

セキュリティ(主題1.10)

クラウドセキュリティの基礎

1. 云计算中最重要的安全概念——责任分担模型（Shared Responsibility Model）。

💡 知识点总结

- **责任分担模型**：云服务商（CSP）负责“**云本身的安全**”（物理机、数据中心），而用户负责“**云内部的安全**”（实例、操作系统、数据）。
- **要塞化（Hardening）**：指通过关闭不必要的端口、更新补丁、设置防火墙规则等手段，提高实例（Instance）安全性的过程。
- **安全组（Security Group）**：云厂商提供的虚拟防火墙，是用户实施安全控制的首要工具。

✅ 正确答案

ファイアウォールなど、サービス事業者が提供する機能でユーザがインスタンスを要塞化する
(用户使用服务商提供的防火墙等功能，对实例进行要塞化/加固)

🔍 选项解析与详细说明

a. 为什么这个描述是正确的？

虽然云厂商提供了基础设施，但他们不知道你的业务需要开放哪些端口。因此，厂商会提供**虚拟防火墙**（如安全组）、IAM（身份管理）等工具，**用户必须主动使用**这些功能来锁定自己的实例。

❌ 错误选项解析

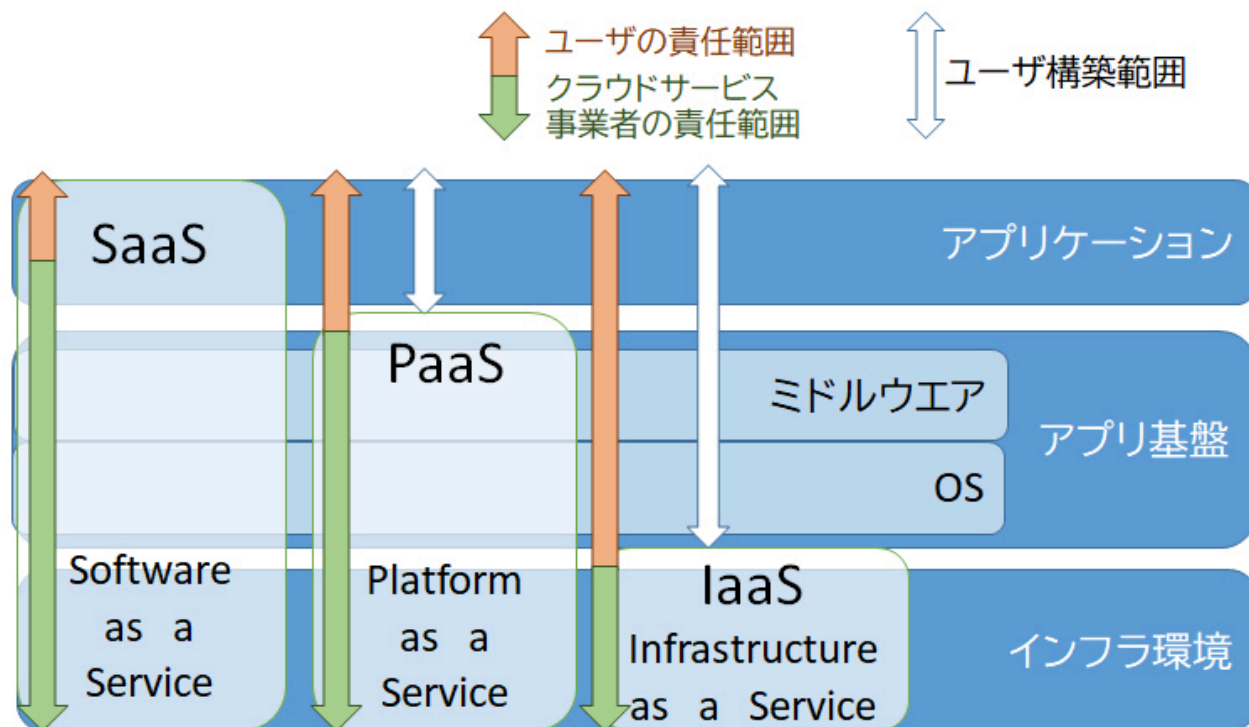
- **セキュリティ対策はクラウド基盤全体へ...**：云基座的安全不代表你的实例安全。如果用户给实例设了弱密码或开放了所有端口，基座再安全也没用。
- **サービス事業者が提供する機能は古いため...**：云厂商提供的安全功能（如 AWS Security Group 或 Azure NSG）通常是非常先进且高度集成的，用户应当优先使用，而非“仅使用”第三方产品。
- **デフォルトの网络设置では通信优先...**：现代主流云厂商的默认设置通常是“**默认拒绝（Deny All）**”，即安全性优先，用户不手动开启端口是无法通信的。
- **ユーザは設定に関与しない**：这完全违反了责任分担模型。用户必须对实例的配置、补丁和权限负责。

想象你在云厂商那里**租了一间公寓**（实例）：

- **物业（服务商）**：负责大楼保安、监控、电梯安全、外墙防漏水。
- **你（用户）**：负责你自己房间的门锁、保险柜。

- **逻辑**：物业会给你配好防盗门和猫眼（防火墙功能），但你必须自己记得锁门。如果你不锁门导致丢了东西，那是你的责任，而不是物业的。

クラウドにおける3つのサービスとセキュリティの責任範囲



📝 考试小秘籍

- 关键词匹配**：看到“云安全”，立刻寻找「ユーザが設定する」（用户设定）或「責任共有」（责任共有）等词汇。
- 默认规则**：记住现代云的默认原则是「最小権限」或「Default Deny」。
- 安全组原则**：记住安全组通常是「ステートフル（Stateful）」的，且遵循「最小権限の原則」。
- 排除法**：任何说“用户不需要管安全”或者“厂商全包了”的选项，统统都是错的。

为了帮你彻底记住这三个“服务边界”，我们利用**责任范围的层级关系**来编几个顺口溜。

核心逻辑是：**层级越低（IaaS），用户管得越多；层级越高（SaaS），用户管得越少。**

IaaS（“唉——呀”式）：

- **谐音**：“哎呀，啥都要管”。
- **内容**：要管OS、要管墙（防火墙）、要管应用。

PaaS（“跑”平台式）：

- **谐音**：“平台‘跑’应用”。
- **内容**：不管底层（OS/网络），只管应用和数据。

SaaS（“傻”瓜式）：

- **谐音**：“傻瓜式使用”。
- **内容**：只管**数据**就行。

2. 这道题考查的是 **パブリッククラウド (公有云)** 的基础设施架构中，最基本的地理概念——**リージョン (Region/区域)**。

💡 知识点总结

- **リージョン (Region)**：指云服务商在全球范围内划分的**地理区域**（如：东京区域、新加坡区域、美东区域）。每个区域由多个互相独立的**可用区 (Availability Zone, AZ)** 组成。
- **地理隔离**：区域之间物理距离较远，旨在降低由于地震、洪水等大规模灾害导致所有数据同时丢失的风险。
- **法律法规**：由于区域分布在不同国家，数据存储在该区域时必须遵守当地的**法律法规**（如欧盟的 GDPR）。

✅ 正确答案

- **リージョンによって法律や制限が異なる場合がある**
- **データセンターが配置されている、地理的に分離された地域のことである**

🔍 选项解析与详细说明

为什么这两个选项正确？

- **地理的分離**：一个 Region 通常包含一个或多个数据中心集群。它们相隔很远，以确保一个地方出事，另一个地方还能正常运行。
- **法律や制限**：这是考试常考点。例如，如果你选择“东京区域”，数据就存储在日本，受日本法律管辖；如果选“法兰克福区域”，则必须遵守极为严格的欧洲隐私法。

❌ 错误选项解析

- **Webブラウザから...管理するツール**：这描述的是 **コンソール (Console)** 或 **ダッシュボード (Dashboard)**，而不是区域。
- **各クラウドサービス事業者のこと**：这指的是 **プロバイダ (Provider)**，如 AWS、GCP、Azure 等。
- **どのリージョンにも同じ法律が適用される**：错误。法律是跟国家挂钩的，不同国家的区域法律完全不同。

想象一家全球连锁酒店（云服务商）：

- **リージョン (Region)**：就是**城市**（比如北京、伦敦、纽约）。你在哪个城市住，就要遵守哪个城市的规定（法律）。
- **可用区 (AZ)**：就是同一个城市里的**不同分店**。如果 A 街的分店停电了，你可以去 B 街的分店住，反正都在同一个城市。

口诀：Region 是城市，法律各不同；物理隔得远，灾难不怕碰。

❌ リージョンには、利用ユーザの事業所がある国の法律や制限が適用される

在法律适用问题上，公有云遵循的是“属地原则”，而不是“用户所属原则”。我们先来看正确答案，再详细解析为什么 4 是错的。

✅ 正确答案

リソースやインスタンスごとに複数のリージョンを選択できる（可以为每个资源或实例选择不同的区域）

选项 4 原文：リージョンには、利用ユーザの事業所がある国の法律や制限が適用される。
(Region 适用的是利用用户所在地国家的法律和限制。)

❌ 错误原因：法律跟“数据在哪”走，不跟“人在哪”走。

- **物理决定法律**：如果你的公司在日本（用户所在地），但你为了便宜把服务器开在了**美国俄勒冈区域（Region）**，那么这台服务器物理上是在美国领土。因此，它主要受**美国法律**（如《美国爱国者法案》）的管辖和限制，而不是日本法律。
- **合规性冲突**：正因为如此，很多金融或政府机构会强制要求“数据不能出境”，就是为了防止数据落入外国法律的管辖范围。

📖 考试小秘籍

- a. **关键词匹配**：看到 Region，脑子里要反射出两个词：**「地理的に隔離」**（地理隔离）和**「所在国の法律」**（所在地法律）。
 - b. **多选策略**：如果题目问 Region 的好处，通常选：**降低延迟 (低レイテンシ)** 和 **灾难恢复 (DR/BCP)**。
 - c. **注意主语**：选项 4 的主语是“用户的所在地”，这是典型的干扰项，一定要看清是**「ユーザの場所」** 还是 **「リージョンの場所」**。
3. SaaS（Software as a Service）は、ソフトウェアをサービスとして提供します。ユーザはインフラも開発環境も意識することなく、単純にソフトウェアを利用する、というサービスを受けることができます。

クラウドにおけるサービスモデルのうち、ソフトウェアをサービスとして提供するものはSaaS
这道题考查的是云计算的三种基本**服务模式（Service Models）**。在 LPI 考试中，区分这三者的“服务边界”是必考的基础知识。

💡 知识点总结

- **SaaS (Software as a Service)：软件即服务**。通过互联网提供现成的应用程序，用户只需登录浏览器即可使用（如 Gmail、Office 365）。
- **PaaS (Platform as a Service)：平台即服务**。为开发者提供开发、运行和管理应用程序的平台（如数据库、中间件），用户无需关心服务器底层。

- **IaaS (Infrastructure as a Service): 基础设施即服务。**提供计算资源（虚拟 CPU、内存、存储、网络）。用户需要自己安装操作系统和软件。
- **オンプレミス (On-premises): 指本地部署/自建机房。**这与云计算是相反的概念，所有硬件软件都要自己买。
- **HaaS (Hardware as a Service): 硬件即服务。**现在基本被包含在 IaaS 概念中，不是主流的三大分类之一。

口诀：I 是基建（硬盘机器），P 是平台（环境工具），S 是软件（直接上手）。

考试小秘籍

a. 关键词匹配：

- 看到「ソフトウェア」（软件）或「アプリケーション」（应用）→ 选 **SaaS**。
- 看到「プラットフォーム」（平台）或「ミドルウェア」（中间件）→ 选 **PaaS**。
- 看到「インフラ」（基础设施）或「仮想サーバ」（虚拟服务器）→ 选 **IaaS**。

b. 管理责任：SaaS 的用户责任最小，IaaS 的用户责任最大。

4. 这道题考查的是云环境中 **ストレージ（存储）** 的基本属性。理解“挥发性”与“不挥发性”的区别，对于设计一个既安全又高性能的系统至关重要。

知识点总结

- **揮発性ストレージ (Volatile Storage / Ephemeral Storage):**
 - **特性：**断电或实例（Instance）停止后，数据会**丢失**。
 - **用途：**通常用作临时缓存（Swap、临时文件），因为它的物理位置通常离 CPU 很近。
- **不揮発性ストレージ (Non-volatile Storage / Persistent Storage):**
 - **特性：**即使实例停止或重启，数据也会**永久保留**。
 - **用途：**存储操作系统（根分区）、数据库、用户文件等。

正确答案

- 揮発性ストレージは不揮発性ストレージよりも I/O パフォーマンスが良い
- 不揮発性ストレージはインスタンスを停止しても永続的にデータを保持する

为什么挥发性存储性能更好？

在云端，挥发性存储（如 **インスタンスストア / Instance Store**）通常是直接挂载在物理主机上的 SSD。因为它不需要通过网络传输数据，所以其 **I/O 速度（读写性能）** 远高于通过网络连接的持久化存储（如 EBS）。

为什么不挥发性存储更安全？

不挥发性存储（如 **EBS, Block Storage**）的设计初衷就是“持久化”。它的寿命独立于实例，即使你把服务器关了，数据依然稳稳地存在云端的磁盘阵列中。

口诀：挥发快如风，停机一场空；持久稳如钟，重启还能用。

6. “实例”这个词听起来很抽象，但如果你把它对应到现实世界，它就非常容易理解了。

简单来说：“实例（Instance）”就是你在云端租用的一台“虚拟电脑”。

我们可以从三个层面来彻底搞懂它：

现实生活中的类比：饭店里的“桌位”

想象云服务商（如 AWS 或 阿里云）是一个巨大的**披萨店**：

- **物理服务器**：是披萨店里那张巨大的、实心的**大圆桌**。
- **实例（Instance）**：是你走进店里，服务员通过活动隔板，在大圆桌上为你**划分出的一个独立座位**。
 - 虽然你和其他人共用一张大圆桌（物理硬件），但你的座位（实例）有自己的椅子、餐具。
 - 别人吃什么、怎么吃，都不会影响你。
 - 当你吃完离开（销毁实例），这个位置（资源）就会被收回，重新分配给下一个人。

这里的“实”到底是什么意思？

在编程或 Linux 领域，“实例”通常指：“**根据模版创造出来的具体实物**”。

- **镜像 (Image)**：相当于这台电脑的“出厂说明书”或“备份光盘”。里面写着：安装了 CentOS 系统、带了 Python 环境。它是死的，不能运行。
- **实例 (Instance)**：当你按下“启动”按钮，云平台按照镜像的要求，在物理机上切出了一块 CPU、分配了 8G 内存。这台**正在运行的、活生生的电脑**，就叫一个“实例”。

7. 这道题考查的是在公有云环境下进行脆弱性扫描（Vulnerability Scanning）**或**渗透测试（Penetration Testing）时的合规性与操作规范。

 **知识点总结**

- **合规性确认**：由于云环境是共享的，未经授权的扫描行为可能会被云服务商（CSP）的入侵检测系统判定为**真实攻击**，从而导致账号被封禁或触发法律风险。
- **责任分担**：虽然云服务商负责底层的安全，但实例内部的漏洞（如未打补丁的软件）仍由用户负责。因此，用户**需要**定期进行漏洞检查。

 **正确答案**

- 脆弱性チェックはプロバイダに了承を得てから行う
- 攻撃と間違われる可能性がある

 **选项解析与详细说明**

为什么要获得许可？

在公有云中，你的服务器可能与成千上万个其他客户的服务器共享物理硬件或网络通道。如果你发起高强度的漏洞扫描，产生的异常流量可能会影响到其他客户，或者触发表方的防御警报。

- **注：**虽然现在像 AWS 等主流厂商已经放宽了对部分常用实例进行漏洞扫描的预先申请要求，但在考试中，“**确认许可/合规性**”仍然是标准答案。

为什么会被误认为攻击？

漏洞扫描通常涉及大量的端口扫描（Port Scanning）和尝试性注入。这些行为在云服务商的监控眼中，与黑客实施攻击前的“踩点”行为完全一致。

✗ 错误选项解析

- **サービスモデルに関わらず、脆弱性チェックは推奨されない：**错误。漏洞检查是安全运维的必要环节，非常被**推荐**，只是需要合规操作。
- **パブリッククラウドの脆弱性についてユーザは対策しなくてもよい：**错误。根据“责任分担模型”，操作系统、中间件和应用程序的漏洞必须由**用户**自行修复。

口诀：漏洞检查要合规，先报物业（厂商）不吃亏；猛然扫描像攻击，误封账号掉眼泪。

考试小秘籍

- 策略术语：**在日语考试中，这类政策通常被称为「**侵入テストポリシー**」（Penetration Testing Policy）。
 - 默认逻辑：**记住公有云是一个“多租户”环境，「**他の利用者に影響を与えない**」（不影响其他用户）是一切操作的前提。
 - 厂商差异：**虽然不同厂商（AWS, GCP, Azure）的细节不同，但“遵守政策”和“防止误判”是通用考点。
8. 管理コンソールは、インスタンス（クラウド上の仮想マシン）の構築やデータベース管理など、クラウド機能を設定・管理するツールです。操作はWebブラウザから行います。

这道题考查的是用户如何与云服务商（CSP）的后台系统进行交互。

知识点总结

- **管理コンソール（Management Console）：**云服务商提供的图形化用户界面（GUI），让用户可以像逛网店一样通过点击鼠标来创建实例、配置网络和管理账单。
- **操作方式：**它是基于 Web 的工具，因此只需要一个 **Web ブラウザ（网页浏览器）** 就可以随时随地访问。

为什么是 Web 浏览器？

云服务的核心价值之一是“随处访问”。为了让用户不需要安装复杂的软件就能管理资源，AWS、Azure、GCP 等厂商都开发了功能强大的 Web 后台。只要你有账号和浏览器，就能在全世界任何地方控制你的“实例”。

✗ 错误选项解析

- **Tera Termのような...クライアントから操作する：**这通常是指通过 **SSH** 登录到已经创建好的 Linux 实例内部进行命令行操作，而不是管理云平台本身。

- **専用端末から操作する**：云服务不需要特殊硬件，用你自己的电脑、平板甚至手机浏览器都能管理。
- **データセンターにあるコンソール画面から操作する**：公有云用户是不允许进入数据中心的，一切操作都是通过互联网远程完成的。

考试小秘籍

- a. **关键词匹配**：看到「管理コンソール」或「ダッシュボード (Dashboard)」，直接找「Web ブラウザ」。
- b. **区分管理与登录**：
 - 管理“云资源”（买机器、删机器）→ **ブラウザ**。
 - 登录“实例内部”（改代码、查日志）→ **SSH / Tera Term**。
- c. **安全性**：虽然浏览器很方便，但在考试中也会提到，为了安全，登录控制台通常需要 **MFA（多因素认证）**。

9. パブリッククラウドの管理コンソールにおける認証の説明として

1. **多段階認証より多要素認証の方が安全性が高い**
2. **多要素認証は、ID/パスワードとトークンのワンタイムパスワードなど、異なる要素の認証の組み合わせである**

10. クラウドにおけるサービスモデルのうち、サーバ、ストレージ、ネットワーク等、仮想化されたインフラ環境をサービスとして提供するものは**IaaS (Infrastructure as a Service)**：**基础设施即服务**。

クラウドにおけるサービスモデルのうち、ソフトウェアの開発、実行環境をサービスとして提供するものは**PaaS**

11. 这道题考查的是在公有云环境中，用户如何应对**服务商（Provider）侧的维护操作**。这直接关系到系统的“高可用性”设计。

知识点总结

- **维护的不可避免性**：云服务商为了修复底层硬件故障、更新物理机补丁或升级网络设备，必须定期进行维护。
- **责任分担**：虽然服务商会提前通知，但**如何处理维护带来的停机风险**（如：设置多台服务器负载均衡）是**用户**的责任。
- **信息获取**：用户需要主动通过服务商提供的「ダッシュボード (Dashboard)」或「ステータスサイト (Status Site)」来获取维护计划。

正确答案

- **インスタンスの停止や再起動により影響があることがある**
- **プロバイダが公開する事前情報や障害情報を把握すべきである**

选项解析与详细说明

为什么“停机和重启”会产生影响？

有些底层维护（比如更换宿主物理机的主板或升级内核）无法在开机状态下完成。此时，云服务商会要求实例**停止（Stop）并重新启动（Start）**。如果你的应用没有设置自动启动，或者数据只存在于“挥发性存储”中，那么维护就会导致业务中断或数据丢失。

为什么必须把握事前情报？

云服务商通常会提前几天甚至几周通过邮件或管理控制台发布「**メンテナンス通知**」。

- **用户的应对**：你可以选择在通知的时间段内手动迁移业务，或者通过架构设计（如多可用区部署）来自动规避风险。

✖ 错误选项解析

- **メンテナンス中もサービスの可用性は保証されている**：错误。维护本身就是对可用性的挑战。虽然云服务商追求 99.9% 以上的可用性，但单台实例在维护期间是**不保证**持续运行的。
- **メンテナンスの実施にはユーザの承認が必要...**：错误。如果是紧急的安全补丁或底层硬件故障，服务商会**强制执行**。你不能因为“不批准”就让服务商不修物理服务器。

セキュリティ管理業務の実施

1. 「last」 コマンドは「/var/log/wtmp」ファイルを参照し、最近ログインしたユーザの一覧を表示するコマンドです。

「/var/log/wtmp」はユーザのログイン/ログアウト情報が記録されているバイナリファイルです。

在 Linux 中，系统会记录所有用户的登录和注销信息。这些信息被存储在特定的二进制文件中，而 `last` 命令就是专门用来读取这些文件的。

- **wtmp**：记录所有成功的登录和注销历史（从文件创建那一刻起的所有记录）。
- **特征**：它是二进制文件，不能直接用 `cat` 或 `vi` 查看，必须通过 `last` 命令解析。

口诀：过去登录看 wtmp，现在谁在看 utmp，坏人登录看 btmp。

文件路径	记录内容	查看命令	记忆点
/var/log/wtmp	历史登录/注销记录	last	What Time (什么时候登录过)
/var/run/utmp	当前正在登录的用户	who, w, users	User (现在谁在用)
/var/log/btmp	失败的登录尝试	lastb	Bad (坏的登录尝试)

- 最近ログインしたユーザの一覧を表示するコマンドは「last」 コマンド
- 再起動の履歴を確認できるコマンドは「last」 コマンド

2. 「w」と「who」 コマンドは、「/var/run/utmp」ファイルを参照し、現在システムにログインしているユーザの情報を表示します。「/var/run/utmp」ファイルはログイン中のユーザの情報が格納されているバイナリファイルです。

在 Linux 运维中，监控谁正在系统上执行操作非常重要。这些信息通常从二进制文件 `/var/run/utmp` 中读取。

- **who**：显示当前登录用户的基本信息（用户名、终端、登录时间、来源 IP）。输出简洁，适合脚本处理或快速查看在线人数。
- **w**： `who` 的增强版，不仅显示谁在登录，还显示他们**正在干什么**（当前运行的进程）以及系统的负载情况。输出最详细。它第一行会显示 `uptime`（负载），后面列出每个用户的 TTY、登录时长、闲置时间（IDLE）以及 **WHAT**（当前命令）。
- **utmp 文件**：它是记录当前登录状态的核心文件，上述命令本质上都是它的前端展示工具。

口诀：想知谁在 who 一下，想知干啥 w 查。

多选陷阱：有时候选项里会出现 `last`。记住：`last` 是看过去的历史 (`wtmp`)，而 `who/w` 是看现在的状态 (`utmp`)。

3. sudo コマンドの設定を行うため、「/etc/sudoers」ファイルを編集したい。この時使うコマンドはvisudo

这道题考查的是 Linux 系统中针对 `sudo` 权限配置文件的安全编辑方式。

💡 知识点总结

- `/etc/sudoers`：这是定义哪些用户可以用 `sudo` 执行哪些命令的核心配置文件。
- **visudo**：这是专门用来编辑 `/etc/sudoers` 的命令。
- 为什么要用 visudo 而不是直接用 vi？
 - i. 语法检查：`visudo` 在保存退出时会检查语法。如果写错了（比如少了个空格），它会提醒你，防止你因为配置错误而导致整个系统的 `sudo` 功能瘫痪。
 - ii. 锁定机制：防止多个人同时编辑同一个文件造成冲突。

4. sudo コマンドの設定を行うファイル名/etc/sudoers

5. 一般ユーザーが、自分に権限委譲されているコマンドを調べたい時に使うコマンドはsudo -l

「sudo」コマンドで、一般ユーザでも、管理者から実行権限を委譲されたコマンドであれば実行できます。自身に委譲されているコマンドは「sudo -l」で確認できます。

参数	功能	记忆点
sudo -l	列出 (List) 当前用户拥有的特权命令	List = List
sudo -v	更新缓存的时间戳（延长有效期）	Validate (校验)
sudo -u	指定以某个用户身份执行（默认 root）	User
sudo -k	清除时间戳缓存，下次执行必须输密码	Kill (杀掉缓存)

为什么是 `sudo -l`？

`-l` 代表 **list**。当你运行这个命令时，系统会提示你输入**自己的密码**，验证通过后，它会显示类似于：`User user01 may run the following commands on this host: (ALL) /usr/bin/apt, /usr/bin/systemctl` 这清晰地告诉你，你被允许执行哪些命令。

6. 这道题考查的是如何查看特定端口（Network Port）正在被哪个进程占用。在 Linux 安全排查中，这是识别可疑服务或确认服务启动状态的必备技能。

💡 知识点总结

- **lsof (List Open Files)**：在 Linux 中“万物皆文件”，网络连接也被视为文件。`lsof` 是查看进程打开了哪些文件的强大工具。
- **-i 参数**：专门用于显示网络相关的文件（Internet files）。
- **语法**：`lsof -i:[端口号]` 可以直接锁定特定端口。

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE	NODE	NAME
sshd	2462	root	3u	IPv6	6988			TCP *:ssh (LISTEN)

以下の表示結果はポート22番（ssh）を使用しているプロセスを表示している/图片中展示的表头（COMMAND, PID, USER, FD, TYPE, DEVICE, SIZE, NODE, NAME）是 lsof 命令的标志性输出格式。

- **端口 25**：这是标准的 **SMTP**（邮件传输协议）端口。通过检查它，可以确认邮件服务器（如 Postfix 或 Sendmail）是否正在运行。

为什么是 `lsof -i:25`？

这个命令会输出非常直观的信息，包括：

- **COMMAND**：进程的名字（如 `master`）。
- **PID**：进程 ID。
- **USER**：运行该进程的用户。
- **NODE**：协议类型（TCP/UDP）。
- **NAME**：监听的地址和状态（LISTEN）。

口诀：查端口，用 lsof，减 i 加个冒号，进程全跑不掉。

📝 考试小秘籍

- 权限注意**：普通用户可能看不到其他用户（如 root）启动的进程端口信息，所以通常需要配合 `sudo lsof -i:25` 使用。
- 协议区分**：如果你只想看 TCP 协议的 25 端口，可以用 `lsof -iTCP:25`。
- 替代方案**：如果在考试中没看到 `lsof`，记得找 `ss` 或 `netstat` 配合 `grep` 的选项。

7. chage コマンド

这道题考查的是 Linux 系统中专门用于管理用户密码过期策略（Password Aging）的工具。

💡 知识点总结

- `/etc/shadow`：这是 Linux 存储加密密码和密码有效期信息的敏感文件。为了安全，普通用户无法查看或编辑。
- **chage (change age)**：这是修改用户密码过期信息的最专用、最安全的命令。它可以修改诸如“下次登录必须改密码”、“密码最长有效期”、“过期前提醒天数”等细节。
- **与 usermod 的区别**：虽然 `usermod` 也能修改一部分有效期信息，但 `chage` 提供了更完整、更交互式的管理功能（如 `chage -l` 查看详情）。

为什么 `chage` 最合适？

`chage` 的设计初衷就是为了管理“时间（Age）”。

- **常用参数：**
 - `-l`：列出当前的所有有效期详情。
 - `-M`：设置密码最长使用天数（Max days）。
 - `-d 0`：强制用户下次登录时立即修改密码。

口诀：密码过期（Age）不要慌，找准 chage 帮大忙。

考试小秘籍

- 路径记忆**：记住有效期信息存在 `/etc/shadow`，而基本用户信息存在 `/etc/passwd`。
- 强制改密考点**：考试中经常问“如何让用户下次登录强制改密码”，答案通常是 `chage -d 0 [用户名]`。
- 安全性**：由于 `/etc/shadow` 权限极高，运行 `chage` 通常需要 root 权限（即 `sudo chage`）。

書式：chage [オプション] ユーザ名

chage コマンドのオプション

[]内はオプションが表している文字

なし	対話モード(ユーザ名の指定は必要)
-l	パスワードの有効期限に関する情報を表示 [ls]
-m	パスワード変更の間隔の最短日数を指定 [minimum] (パスワードを変更出来るようになるまでの日数)
-M	パスワードが有効な最長日数を指定 [Max] (何日ごとに必ずパスワードを変更させるか)
-I	パスワードの有効期限切れ後にアカウントがロックされるまでの日数を指定 [Inactive]
-W	パスワードの有効期限が切れる前に警告を表示する日数を指定 [Warning] (有効期限の何日前から警告を行うか)
-E	ユーザアカウントの有効期限を指定 [Expire] (YYYY-MM-DD形式)

选项	记忆逻辑 (英文单词对照)	详细功能说明 (日语对照)
-l	List (列表) / ls	表示 [ls]: 查看当前密码的所有“保质期”信息。
-m	minimum (最小)	最短日数: 改完密码后, 至少要等几天才能再次修改。
-M	Max (最大)	最長日数: 密码最长能用几天。到期必须强制修改。
-W	Warning (警告)	警告表示: 保质期快到时, 提前几天开始发通知。
-I	Inactive (不活跃)	ロックされる: 过期后如果不改, 再过几天账号彻底被锁死。
-E	Expire (到期)	アカウント有効期限: 设置整个账号在哪一天彻底报废。

处理“密码过期”问题的排查表

如果你在考试中遇到情景题，可以参考这个逻辑顺序：

- 想看信息？ → `chage -l`
- 强制明天改？ → `chage -M 1` (或者 `chage -d 0`)
- 防止用户乱改？ → `chage -m [天数]`
- 到期前提醒？ → `chage -W [天数]`

“列表小 l，长短两个 M；W 警告 I 锁死，E 是到期年月日。”

パスワードの有効期限に関する設定が可能なコマンドは、「chage」、「passwd」、「usermod」コマンドです。

考试小秘籍

- 大小写区别：考试最喜欢考 `-m` 和 `-M`。记住：大 M 管上限 (Max)，小 m 管下限 (min)。

- b. **交互模式**：如果你什么参数都不加，只输入 `chage [用户名]`，它会进入「対話モード」，一个一个问你。
- c. **日期格式**：`-E` 后面接的是具体的日子（如 2026-12-31），而其他参数后面通常接的是「日数」（如 90 天）。



例题:chageコマンドのオプションのうち、root以外の一般ユーザが実行できるものは-l

这道题考查的是 **chage 命令的权限限制**。在 Linux 系统中，普通用户对自己账号的“保质期”管理权限是非常有限的，以防止用户通过修改策略来逃避密码过期的限制。

在 `chage` 命令的众多选项中，绝大多数（如修改到期时间、最短/最长有效期等）都属于**管理行为**，必须由 **root** 用户执行。

- **普通用户**：仅能查看（List）自己的密码过期信息。
- **root 用户**：可以修改任何选项（`-M`，`-m`，`-W`，`-E`，`-I` 等），也可以查看任何人的信息。

为什么只有 `-l` ？

- **-l (list)**：普通用户运行 `chage -l` 可以看到自己的密码最后修改时间、有效期、警告天数等信息。这被认为是用户的“知情权”，所以不需要 root 权限。
- **执行方式**：一般用户执行 `chage -l [自己的用户名]`。

ユーザ「user1」のパスワードをいつでも変更できるようにしたい。👉 **`chage -m 0 user1`**

8. 这道题考查的是 Linux 中用于管理系统资源限制（Resource Limits）的内置命令 `ulimit`。

💡 知识点总结

- **ulimit**：这是 Shell 的内置命令，用于控制 Shell 及其启动的进程所能消耗的各种资源（如打开的文件数、内存大小、创建的文件大小等）。
- **-a (All)**：这个参数的作用是「**一覧表示**」（显示所有当前的限制值）。

当执行 `ulimit -a` 时，系统会列出所有的资源限制项，包括最大文件大小、最大进程数、管道大小、栈大小等。输出结果通常如下所示：

- `core file size`
- `data seg size`
- `open files`
- `max user processes`

書式：`ulimit [オプション] [制限値]`

オプション	説明
-a	リソースの制限値をすべて表示(all)
-c	コアファイルのサイズを制限(core)
-f	出力ファイルのサイズを制限(file)
-u	ユーザー1人が起動できるプロセス数を制限(user process)
-v	シェルが利用できる仮想メモリを制限(virtual memory)

小 a 全部看一看，小 c 核心（core）不能偏。小 f 文件（file）设上限，小 u 进程（user）管得严。小 v 虚拟（virtual）メモリ限制全。

这组参数非常好记，因为它们几乎都是对应英文单词的首字母：

- a → All（全部显示）
- c → Core（Core 文件，核心文件）
- f → File（文件大小）
- u → User process（用户进程数）
- v → Virtual memory（虚拟内存）

a 是全部，c 是核心，f 是文件，u 是进程，v 是虚存。

 考试小秘籍

a. 关键词锁定：考试中如果出现「すべて表示」或者是「一覧表示」，百分之百选 `-a`。

b. 区分 f 和 c：

- f 是你平时保存的普通文件（File）。
- c 是程序报错崩溃后留下的“遗骸”文件（Core）。

c. 单位注意：在 Linux 系统中，`ulimit -f` 的单位通常是 **blocks**，而 `ulimit -v` 的单位通常是 **kbytes**。

9. 指定した時間、何も入力が無ければ自動的にログアウトさせる環境変数は**TMOUT**

10. `passwd [オプション] [ユーザ名]`

`passwd` コマンドのオプション

オプション	説明
-l	パスワードをロックする
-u	パスワードをアンロック(解除)する
-x	パスワードの有効期限を変更する

l 锁 u 开，x 设期。

11. 这道题考查的是如何处理正在占用特定文件的可疑进程。题目要求同时实现三个动作：确认PID（进程ID）、确认用户信息、以及强制终止（Kill）进程。

💡 知识点总结

- **fuser (File User)**: 这是一个非常强大的命令，用于识别哪些进程正在使用某个文件、目录或套接字。
- **功能组合**:
 - **显示 PID**: `fuser` 的默认功能。
 - **-u (User)**: 显示进程所有者的**用户名**。
 - **-k (Kill)**: 向占用该文件的进程发送信号（默认是 SIGKILL），从而**终止进程**。
- **联动效应**: `fuser -uk` 可以一气呵成地完成“查人”和“杀进程”的动作。

fuser [オプション] 検索対象

オプション	説明
-k	プロセスに SIGKILL を送信 (kill) 送りたいシグナルを指定するには -kではなく「-シグナル名」とする
-m	マウントされたファイルシステムを 指定 (mount)
-n	検索する領域 (名前空間) を指定 (namespace) file (ファイル名: デフォルト)、udp (UDPポート番号)、tcp (TCPポート 番号) が指定できる
-u	検索結果にユーザー名を追加 (user)
-v	詳細な情報を表示 (verbose)

小 k 杀 (Kill)，小 m 挂 (Mount)。

小 n 定空间 (Namespace)，小 u 加入 (User) 忙。

详细信息看小 v (Verbose)，谁占文件都别藏。

选项	记忆逻辑 (英文单词对照)	详细功能说明 (日语对照)
-k	Kill (杀死)	SIGKILL を送信: 向进程发送杀掉信号，强制终止它。
-m	Mount (挂载)	マウントされたファイルシステム: 指定已挂载的文件系统（如磁盘）。
-n	Namespace (空间)	名前空間を指定: 指定搜索领域，如 file、udp 或 tcp。
-u	User (用户)	ユーザー名を追加: 在显示结果中加入进程所属的用户名。
-v	Verbose (冗长)	詳細情報を表示: 显示非常详细的信息（类似表格形式）。

k 是杀手，m 挂载；n 找空间，u 加入；详细信息小 v 准。



例题；fuserコマンドで、/mnt/dvdにマウント中のDVD-ROMにアクセスしているプロセスのプロセスIDを特定したい。適切なオプションは-m

-u (User)：它是用来“增加显示内容”的。

- 默认情况下 `fuser` 只显示 PID（数字）。
- 加了 `-u`，它会在 PID 后面额外加上用户名（比如 `root` 或 `user01`）。
- **结论**：它不决定“搜哪里”，只决定“结果里显不显示名字”。

-m (Mount)：它是用来“确定搜索范围”的。

- 如果你直接查一个目录（不加 `-m`），它只看“正在读写这个目录本身”的进程。
- 如果加了 `-m`，它会扫描整个挂载点（整个 DVD-ROM）下的所有文件。
- **结论**：在查 DVD、U 盘、硬盘挂载点时，必须用 `-m` 才能查得彻底。

为什么题目中说“特定进程 ID”却不选 `-u`？

1. **默认值就是 PID**：即便你不加任何选项执行 `fuser /mnt/dvd`，它的默认输出就是进程 ID（PID）。所以你不需要为了得到 PID 而特意加 `-u`。
2. **-m 的必要性**：DVD 里面可能有很多文件夹和文件。如果有进程正在读 DVD 里的 `/mnt/dvd/movie.mp4`，普通的 `fuser /mnt/dvd` 是抓不到它的。只有加上 `-m`，`fuser` 才会明白：“哦！你是要查整个 DVD 设备啊”，然后把 DVD 里所有文件的占用者都揪出来。



例题2：fuserコマンドで、リソースを使用中のプロセスの実行ユーザーを特定したい。適切なオプションは-v、-u

这道题考查的是如何通过 `fuser` 获取“用户（User）”信息。在考试中，当你看到“特定（Identify）”或“表示（Display）”执行用户时，通常有两个途径可以实现。

-u (User)：在输出结果中直接追加（Add）用户名。它会让 PID 后面跟着括号，里面写着用户名字。

-v (Verbose)：开启详细模式（Detailed）。它会以表格的形式展示信息，其中明确包含了一列名为 `USER` 的字段。

默认情况：如果不加任何选项，`fuser` 仅显示进程 ID（PID），无法直接看到用户名。

12. 这道题考查的是 `find` 命令的类型过滤（Type filtering）。在 Linux 系统中，每一个文件都有其特定的属性类型，`find` 命令通过 `-type` 参数来锁定这些属性。

find 命令：Linux 中功能最强大的搜索工具。

-type 参数：专门用于根据文件类型进行筛选。

l (Link): 在 Linux 文件类型简写中，`l` 代表 **Symbolic Link (シンボリックリンク / 软链接)**。

为什么定期检查很重要？：黑客有时会创建指向系统敏感文件（如 `/etc/shadow`）的软链接，以此尝试越权访问。

为什么选择 `-type l`？

- `find` 命令预设了字母缩写来代表不同的文件类型。
- 输入 `find / -type l`，系统就会列出根目录下所有的软链接文件。

`find` 搜寻定类型，首字母里显本领。软链选 `l` (Link) 不选 `f` (File)，目录选 `d` (Directory) 最分明

区分 `l` 和 `f`：这是最高频的考点。记住：`l` 是虚的（链接），`f` 是实的（文件）。

13. `lsuf` コマンドは、ファイルシステムやポートなど、システムのリソースを使用しているプロセスの情報を表示できます。

`/mnt`をアンマウントしようとしたところ、リソースが使用中のためアンマウントできなかった。
使用中のプロセスを特定することができるコマンドは**`lsuf`**

14. `w`や**`who`**コマンドで参照するバイナリファイルは**`/var/run/utmp`**

15. 这道题考查的是使用 `find` 命令根据特殊权限（SUID/SGID）进行精确搜索的方法。
在 Linux 权限系统中，除了基础的 `rwX`，还有三种特殊权限：

- **SUID (Set User ID)**：权限数值为 **4000**。执行文件时，进程会以**文件所有者**的身份运行。
- **SGID (Set Group ID)**：权限数值为 **2000**。执行文件时，进程会以**文件所属组**的身份运行。
- **两者结合**：如果一个文件同时设置了 SUID 和 SGID，权限总和就是 $4000 + 2000 = 6000$ 。

在 `find -perm` 命令中，数字前的符号非常关键：

- **- (连字符)**：表示“必须全部包含”。`-6000` 意味着该文件必须同时具备 4000 (SUID) 和 2000 (SGID)。这完全符合题目要求。
- **/ (斜杠)**：表示“包含其中任意一个”。如果选 `/6000`，则会搜出“只有 SUID”、“只有 SGID”或“两者都有”的所有文件。

表达式	匹配逻辑	形象理解
<code>find -perm 6000</code>	精确匹配	权限必须正好是 6000，多一点少一点都不行。
<code>find -perm -6000</code>	包含匹配	权限里必须包含 4000 和 2000（这是我们要的）。
<code>find -perm /6000</code>	或匹配	权限里只要有 4000 或 2000 其中的一个就行。

精确找位不带号，包含匹配减号靠；只要其一用斜杠，六千（6000）双拼最牢靠。

16. 下記の結果を得るコマンド名を答えなさい。

```
Last password change          : 4月 03, 2010
Password expires              : never
Password inactive             : never
Account expires               : 4月 20, 2010
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
```

这道题考查的是如何查看 Linux 用户的**密码过期策略详细列表**

chage -l: 这是最常用的查看用户密码生命周期详情的命令。 `-l` 代表 **List**。

输出内容: 正如你提供的图片所示, 它会列出:

- **Last password change**: 最后一次修改密码的日期。
- **Password expires**: 密码过期的日期 (never 表示永不过期)。
- **Account expires**: 账号失效的日期。
- **Minimum/Maximum days...**: 修改密码的最短/最长间隔天数。

权限说明: 普通用户可以对自己运行此命令, 但查看他人信息需要 **root** 权限。

在考试中, 只要看到带有 **「Last password change」** 这种多行详细列表的截图, 闭着眼选 **chage -l**。

17. su (Switch User) コマンドを使うと、ログイン中に別のユーザーに切り替わる事ができます。

suコマンドには以下の特徴があります。

- 引数に「-」をつけると、切り替わる先のユーザの環境変数を使用する
- ユーザー名を指定しない場合、rootユーザへ切り替わる
- ユーザを切り替える時に求められるパスワードは、切り替わる先のユーザのパスワード
- 但し、rootユーザから別のユーザに切り替わる時はパスワードは要求されない

su (Switch User) 命令的权限验证逻辑和环境变量处理。这是 Linux 系统管理中最基础、也最容易混淆的考点之一。

身份切换: `su` 允许用户在不注销当前会话的情况下, 变为另一个用户。

密码规则:

- **一般用户** → **别人**: 必须输入 **「目标用户 (切り替わる先)」** 的密码。这是为了证明你有权访问那个账户。
- **root** → **别人**: **不需要密码**。因为超级管理员拥有最高权限, 不需要向任何人证明自己。

环境变量 (`-` 或 `-l`):

- 如果不加 `-`: 保留原用户的环境变量 (就像是你穿着自己的衣服进了别人的家)。

- 如果加了 `-`：加载目标用户的环境变量（就像是你换上了别人的衣服住在别人家，完全模拟登录）。

誤った説明:

- ユーザを切り替える時に求められるパスワードは、suコマンドを実行したユーザのパスワードである
- 引数に「-」をつけると、suコマンドを実行したユーザの環境変数を引き継ぐ

为什么“执行者的密码”是错误的？

- 正确逻辑：`su` 命令验证的是你“想变成谁”。如果你想变成 `user01`，你必须输入 `user01` 的密码。
- 混淆点：容易和 `sudo` 搞混。执行 `sudo` 时输入的是「执行者自己」的密码。

为什么“带 - 继承原环境”是错误的？

- 正确逻辑：加上 `-`（或 `--login`）意味着完全登录。它会重置环境，使用目标用户的路径（PATH）、家目录等。

引数「-」：就像是一个**“行李箱”**。

- 如果不带箱子（不加 `-`），你变身之后还穿着自己的脏衣服（旧环境）。
- 如果带上箱子（加上 `-`），你变身后就会换上对方整洁的衣服（新环境）。
- 实际差异：
 - `su root`：你还是在原来的目录下，环境变量没变。
 - `su - root`：你会直接跳到 `/root` 目录，且环境变量完全变成 root 的。

命令	谁要变？	验证谁的密码？	备注
<code>su userB</code>	UserA → UserB	UserB 的密码	必须知道对方密码，不够安全
<code>sudo -u userB</code>	UserA → UserB	UserA 的密码	只要管理员给你授权，用你自己的密码即可
<code>su -</code>	UserA → root	root 的密码	需要知道终极密码

口诀：su 变身看对方，减号 (-) 带走新衣裳；root 切换不设防，sudo 才输自己帐。

 考试小秘籍

- a. 关键词排查：看到 `su` 找「切り替わる先」。看到 `sudo` 找「実行ユーザー自身」。
- b. 默认目标：记住 `su` 后面如果不接用户名，默认就是 `su root`。
- c. 安全性建议：在生产环境中，为了不泄露 root 密码，通常推荐使用 `sudo` 替代 `su`。

18. nmapコマンドは指定したホストの開いているポートをネットワーク経由で確認（ポートスキャン）する事ができます。ホストにログインせずに、外部からアクセス可能なポートを調査します。

```
# nmap -sS 192.168.66.91

Starting Nmap 6.40 ( http://nmap.org ) at 2024-04-11 13:51 JST
Nmap scan report for 192.168.66.91
Host is up (0.0018s latency).
Not shown: 997 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
2049/tcp  closed nfs
9090/tcp  closed zeus-admin
MAC Address: 08:00:27:F7:B6:20 (Cadmus Computer Systems)

Nmap done: 1 IP address (1 host up) scanned in 8.63 seconds
```

PORT	STATE	SERVICE
22/tcp	open	ssh
53/tcp	open	domain
80/tcp	open	http
110/tcp	open	pop3

nmap（网络扫描工具）的典型输出格式识别

- 1. **nmap (Network Mapper)**: 这是一个用于网络探测和安全审核的开源工具。它通过向目标主机发送特定数据包，来确定哪些端口是开放的（Open）、正在运行什么服务（Service）。
- 2. **识别输出格式**: **nmap** 的标准扫描结果通常包含三列：**PORT**（端口号/协议）、**STATE**（状态，如 open）、**SERVICE**（服务名称，如 ssh, http）。
- 3. **与其他命令的区别**:
 - **netstat / ss**：侧重于显示本地的连接状态（如 **Recv-Q**，**Send-Q**）。
 - **chage**：侧重于显示用户账号的过期信息。

nmapコマンドによるTCP SYNスキャンについて,サーバから検出されにくく、ステルススキャンとも呼ばれる（不易被服务器发现，也被称为ステルススキャン/隐身扫描）

这道题考查的是 **nmap** 中最常用的一种扫描技术：**TCP SYN スキャン**（也称为“半开放扫描”或“隐身扫描”）。

- **TCP SYN スキャン (-sS)**: 这是 **nmap** 的默认扫描方式（如果拥有 root 权限）。
- **半开放 (Half-Open)**: 它不会完成完整的 TCP 三次握手。它发送一个 **SYN** 包，如果收到 **SYN/ACK**，说明端口开放；此时它会立刻发送一个 **RST**（重置）包断开连接，而不是发送 ACK。
- **ステルス (Stealth)**: 因为它没有建立完整的连接，所以很多陈旧的应用程序日志里不会记录这次访问，具有一定的隐蔽性。

为什么它是“隐身”的？

在传统的网络服务（如早期的 Web 服务器或邮件服务器）中，只有当 TCP 连接完全建立（Established）后，应用层才会记录日志。SYN 扫描在最后一步故意“毁约”，不发送最后一个 ACK，导致连接半途而废，从而避开了应用层的日志记录。

19.

ホストのセキュリティ設定

- 1. 在 **xinetd** 配置中，用于指定服务只在某一个特定的网卡接口（インタフェース）或 IP 地址上提供监听服务的参数是 **bind** 和它的法定别名 **interface**。
xinetd 被称为“超级服务器（Extended Internet Services Daemon）”。它的主要工作是帮系统里一堆小服务（比如旧版的 FTP、Telnet 等）看门。

- **默认行为**：如果不做任何限制，`xinetd` 会在服务器的**所有网卡、所有 IP 地址**上敞开大门进行监听。
- **安全锁（本题核心）**：如果你的服务器有两个网卡，一个是连接外网的 `1.1.1.1`，一个是连接内网的 `192.168.1.1`。你出于安全考虑，希望某个敏感服务**只对内网开放**。
- 此时，你就必须在 `xinetd` 的服务大括号里，使用 `bind = 192.168.1.1` 或者 `interface = 192.168.1.1` 来强行把该服务“捆绑”在这块特定的网卡上。

因为很多人记不住 `bind`，而网卡在 Linux 里又叫网络接口（Network Interface），所以官方特意把 `interface` 设定为 `bind` 的**完全等价别名**。在配置文件里写这两个效果一模一样！

🔧 考场正牌参数名	🧠 考场及实战背后的运维大白话	📝 常见的合法配置示例	🔒 考场核心判定日文关键字
👉 <code>bind / interface</code>	服务只在 特定网卡/IP 上营业。	<code>bind = 192.168.1.1</code>	インタフェースのIPアドレスを設定
<code>server</code>	当有人访问时， 去哪里启动 真正的服务程序。	<code>server = /usr/sbin/in.telnetd</code>	デーモンの絶対パス、プログラム
<code>socket_type</code>	这个服务到底是走 TCP 还是 UDP ？	<code>stream (TCP) / dgram (UDP)</code>	ソケットのタイプ (Stream/Dgram)
<code>only_from</code>	安全白名单 。只允许哪些 IP 客户端来访问我。	<code>only_from = 10.0.0.0/24</code>	接続を許可するホスト

2. 这道题考查的是现代 Linux（如 CentOS 7/8、RHEL 等）主流防火墙管理工具 `firewalld` 的**核心运行机制**：默认区域（ゾーン / Zone）机制与即时/永久生效（ランタイム/パーマネント）的区别。

如果不加 `--zone` 参数，`firewall-cmd` 会默认将策略应用在当前系统的「デフォルトのゾーン（默认区域）」上；如果不加 `--permanent` 参数，该配置属于「ランタイム設定（临时即时策略）」，会立刻生效，但一旦系统再启动（重启）或者防火墙重载，设置就会彻底消失。

在 `firewalld` 的设计哲学里，有两大雷打不动的考场核心铁律：

1 铁律一：默认地盘判定（如果没有显式指定 Zone）

- 防火墙有很多个不同安全等级的“地盘”（区域 / Zone），比如 `public`、`internal`、`trusted`。
- 当你敲下命令，却**没有**通过 `--zone=xxxx` 告诉系统你想改哪个地盘时，系统会默认把规则加在当前正在掌权的默认区域（デフォルトのゾーン）上（通常默认是 `public`）。

2 铁律二：临时工与正式工（如果没有加 `--permanent`）

- **临时工模式（Runtime，本题情况）**：命令里没有 `--permanent`。
 - **特点**：立刻生效，不需要重载。但是！它只保存在系统的内存里，只要系统一重启（再起動），或者防火墙一刷新，这临时规则就会当场灰飞烟灭。
- **正式工模式（Permanent）**：命令里加了 `--permanent`。

- **特点：**规则会被写入到硬盘配置文件里。它**不会立刻生效**，必须配合 `firewall-cmd --reload`（重载）才能把硬盘里的规则刷进内存里生效。但它能永久保存，不怕重启。

出题人把“立刻生效但重启必丢”和“不立刻生效但重启不丢”的特性全部颠倒了过来，来故意恶心考生。我们来把这些反向烟雾弹挨个排干：

- **全てのゾーンにhttpサービスが追加される**（第一项）
 - **死因：范围被极端夸大！** Linux 防火墙非常严谨，不可能因为你漏写一个参数就一锅端地去修改系统“所有区域”的策略。没写就只针对“默认区域（Default Zone）”。
 - **「firewall-cmd --reload」で設定を再読み込みさせる必要がある**（第四项）
 - **死因：多此一举，南辕北辙！** 正因为我们输入的是不带 `--permanent` 的临时命令，它已经**立刻在内存里生效了**。如果你这时候手欠去敲 `firewall-cmd --reload`，防火墙会重新从硬盘里读取老配置，结果就是**你刚刚加的 http 临时规则会瞬间被冲刷掉（当场消失）！**
 - **システムを再起動すると設定が反映される**（第五项）
 - **死因：完全说反了！** 这是带 `--permanent` 参数时的特征。不带 `--permanent` 的命令，重启的下场只有四个字：“**配置抹干**”（消える）。
3. 只要系统根目录下存在 `/etc/nologin` 这个特定文件，除 root 之外的所有普通用户（一般ユーザ）在尝试登录时都会被系统强行拒绝（禁止），通常用于系统维护（メンテナンス）期间防止干扰。
- 在日常运维中，如果服务器需要进行重大升级、硬件更换或紧急维护，管理员 root 需要登录进去干活，但此时**绝不希望有任何普通员工（一般用户）突然登录进来提交数据、运行程序**，从而干扰维护工作。
- **/etc/nologin 的拦截原理：** Linux 系统的登录组件（如 PAM 模块）在允许用户登录之前，会雷打不动地先去检查一下 `/etc/nologin` 这个文件是否存在。
 - **如果文件不存在：**正常放行。
 - **如果文件存在：**系统会当场化身铁面判官，对除 root 之外的所有普通用户“一秒劝退”，直接断开连接，禁止登录！
 - **顺带的温馨提示功能：** 这个文件不仅能拦截，还能传话。如果你用 `echo "服务器正在维护，请明天再来" > /etc/nologin` 把这句话写进文件里。普通用户登录被拒绝时，屏幕上就会精准显示出这句提示语，非常人性化。
 - **解除禁令：** 维护完成后，只需要执行 `rm /etc/nologin` 把这个文件删掉，公司所有人就又可以开开心心正常登录了。
4. 从 CentOS 7（以及 RHEL 7）开始，系统默认的防火墙管理服务由原来的 `iptables` 正式升级替换为了支持动态区域管理的 `firewalld`。

5. 为了解决老旧超级服务器 `inetd` 功能单一、缺乏安全控制的缺陷，Linux 后来引入了升级版的超级服务器 `xinetd`，它不仅能为每个服务（サービス毎）独立进行精细化的安全配置，还支持我们在前两题学过的 `bind`、`only_from` 等高级安全参数。

在 Linux 家族里，超级守护进程经历了一次重大的换届选举：

- **旧时代 `inetd`（大锅饭时代）**：它是最古老的超级服务器。它所有的子服务（如 ftp, telnet）全部密密麻麻地挤在唯一的一个配置文件 `/etc/inetd.conf` 里面。它就像一个简陋的传达室大爷，只管帮小弟们接听电话（监听端口），没有任何安全防护功能。谁想进来、从哪进来，它一概管不着。
- **新时代 `xinetd`（独立包间时代，本题核心）**：它的英文全称是 eXtended Internet Services Daemon（扩展的网络服务守护进程）。它最大的革命性改进就是“服务独立细化管理”它不仅拥有总控文件 `/etc/xinetd.conf`，还拥有专门的文件夹 `/etc/xinetd.d/`。
 - 每一个子服务（比如 tftp）都在这个文件夹里拥有一个属于自己的独立配置文件（独立包间）。
 - 在这个独立配置文件里，管理员可以像写小作文一样，精细地为这个服务单独配置“只允许哪些 IP 访问（`only_from`）”、“绑定在哪个网卡上（`bind`）”、“最多允许多少人同时在线（`instances`）”等高级安全策略。

6. firewall-cmd コマンドで、「work」ゾーンに http サービスを追加して許可したい。

`firewall-cmd --zone=work --add-service=http`

7. 关于 `firewall-cmd` 查账选项的考点，如果想查看某个区域（或者默认区域）内当前开放了哪些服务、端口等“全部具体规则”，使用的正统选项是 `--list-all`。

在 `firewalld` 的世界里，凡是想要“查看、列出”规则，命令的开头一律要使用 `--list-`。而这一题的核心就在于“看名字猜功能”：

1 `--list-all` 为什么是标准答案？

- **运维大白话场景**：“我想看看我现在的默认区域里，到底放行了 HTTP 没有？开通了 8080 端口没有？”
- **屏幕吐出的内容**：它会把指定区域里的网卡接口（interfaces）、开放的服务（services）、暴露的端口（ports）等具体规则，文字通り（字面意思上）全部（All）拉个清单（List）打印出来。

2 什么时候用 `--list-zones`？

- **运维大白话场景**：“这台服务器上总共有多少个区域（Zone）可以用啊？有没有 dmz 区域？”
- **屏幕吐出的内容**：它只打印区域的名字（如 `block dmz drop external home internal public...`），它只负责列出“地盘的名称”，但根本不会显示每个地盘里面的具体规则。

📊 处理“设备忙”问题的排查表（`firewall-cmd` 高频查看系选项对齐表）

出题人非常喜欢用 `show` 或者单走的 `all` 来伪造选项。请直接背下这张“防火墙查账死锁表”，在考场上进行降维打击：

🔥 考场正牌选项	🧠 考场背后的运维大白话	📝 敲完后屏幕上吐出什么？	🔒 考场核心判定日
👉 <code>--list-all</code> （本题）	“把这个区域里所有的具体规则内容全列出来！”	服务名、端口号、网卡绑定等详情	指定したゾーンの設
<code>--list-zones</code>	“把系统里所有区域的名字列个清单。”	一堆区域的名称（横着排一列）	すべてのゾーンの
<code>--get-default-zone</code>	“告诉我当前谁在掌权（默认区域是谁）？”	public（只吐出一行默认区域名）	デフォルトのゾーン
<code>--state</code>	“防火墙现在是活着还是死了（运行状态）？”	running（运行中）或 not running	ファイアウォールの止か）

8. `firewalld` 默认提供了 9 个不同安全级别的标准区域，其中高频必考的包括绝对信任的 `trusted`、默认当班的 `public`、面向内网的 `internal`，以及直接丢弃数据包的 `drop`。而 `accept` 和 `reject` 是防火墙的“动作/靶向策略（Target）”，绝不是区域的名字！

在 `firewalld` 的设计里，区域（Zone）就像是服务器针对不同网络环境划分出的“治安管理区”。系统默认准备好了 9 种不同治安等级的模板。

我们来看看这三个正确选项在系统里的标准人设：

- a. `public`（公共区域）：系统默认的掌权人。如果你不改设置，所有网卡默认都归它管。它只允许别人访问你的 SSH 或 DHCP，对其他不认识的流量充满警惕。
 - b. `internal`（内部区域）：公司内网专属。当你把网卡绑定到这个区域时，系统会认为对端都是自己人（内网同学），因此会默认放开更多的服务（如 Samba 共享、DNS 等）。
 - c. `drop`（丢弃区域）：最高冷、最硬核的防御区。任何送进这个区域的数据包，系统都会直接无情地丢进垃圾桶（Drop），并且绝不给发送方任何回应，让对方以为你的服务器彻底死机了。
9. `firewalld` 防火墙服务的专属命令行管理工具是 `firewall-cmd`。我们前面学过的所有关于区域、服务、端口的查看与修改，全部都需要通过这个命令来传达。

我们在前两题提到过，`firewalld` 是一个在后台默默运行的“守护进程（Daemon）”。作为一个高冷的后台服务，它本身是不直接和管理员在终端里对话的。

- 代言人 —— `firewall-cmd`：为了让管理员能够下达指令，官方特意为它配了一个专门的命令行客户端（Command-line tool），也就是 `firewall-cmd`（这里的 `cmd` 就是 `command` 的缩写）。
- 工作模式：你在终端里敲下如 `# firewall-cmd --add-service=http` 时，`firewall-cmd` 就会立刻把你的意图打包发给后台的 `firewalld` 服务，由 `firewalld` 最终刷到底层的内核中去。它们两个是“秤砣不离秤杆”的黄金搭档。

10. 现代 Linux（如 CentOS 7/8、Ubuntu 16.04+、RHEL 7+ 等）中，作为系统第一号总管 **systemd** 的正统、全能型命令行控制工具（コマンド）。在采用 **systemd** 的系统上，负责全面掌管服务的启动（起動）、停止、状态确认（ステータス確認）以及开机自启（自動起動）等所有日常运维动作的唯一正统总管命令是 **systemctl**。
11. 在旧时代的 RHEL 6 系系统中，负责控制和查看各种服务开机是否自启的命令是 **chkconfig**。它通过管理 0 到 6 的运行级别（Runlevel）来决定服务的命运。

在 CentOS 6 / RHEL 6 的旧时代，系统初始化不叫 **systemd**，而是叫 **SysVinit**（或 **init**）。那个时候没有 **systemctl** 这个全能王，旧时代的规则非常讲究“各司其职、各管一段”：

- **只管临时开关** —— **service 命令**：在旧时代，如果你想让服务临时启动或停止，必须用 **service** 命令（比如 `# service sshd start`）。但是，它是个“临时工”，管得了今天管不了明天，系统一重启它就失效了。
 - **systemdが採用される前のRHEL 6系ディストリビューションにおいて、各種サービスをその場で起動・停止したり、ステータスの確認を行うコマンドはservice**
 - **专门管开机自启（正式工）** —— **chkconfig 命令（本题核心）**：如果你想在硬盘上刻下死规矩：“以后每次开机，这个服务都必须自己启动！”你就必须去呼叫 **chkconfig**（它的英文全称就是 **check kernel configuration**，检查内核配置）。
 - 比如著名的开机自启命令：`# chkconfig sshd on`
 - 它的核心工作是去打理 `0 / 1 / 2 / 3 / 4 / 5 / 6` 这 7 个运行级别（Runlevel）下的软链接。
12. 这道题考查的是超级守护进程 **xinetd** 的各个服务配置文件（通常位于 `/etc/xinetd.d/` 目录下）中，用来控制该服务“是否禁用（开启/关闭）”的核心开关参数 **disable** 的填空语法。在 **xinetd** 中，**disable** 的意思是“是否禁用”。因此，如果要“利用服务（开启服务）”，需要填写 **disable = no**（不禁用）；如果要“不利用服务（关闭/禁用服务）”，则必须写成 **disable = yes**（禁用）。
13. **xinetd** 为了实现各个服务的精细化独立管理，除了主配置文件 `/etc/xinetd.conf` 之外，专门存放“每一个服务独立配置文件（サービスごとの設定ファイル）”的官方标准目录是 `/etc/xinetd.d/`。

在传统的 Linux 或者是老旧的 **inetd** 时代，所有人都要挤在单个大文件里，修改起来非常不方便。而现代的 Linux 服务普遍采用了“**主配置文件 + .d 文件夹**”的黄金架构：

- **主控文件**：`/etc/xinetd.conf`。这里面只写一些全局的、全公司通用的规矩。
- **独立包间（本题核心）**：`/etc/xinetd.d/`。
 - 这里的 **.d** 是英文 **Directory**（目录）的缩写。
 - **xinetd** 启动时，会自动跑到这个文件夹下面，把里面存放的所有小服务的配置文件（比如 **telnet**、**tftp** 等）全部加载进来。

- 每个服务各占一个文件，想改谁就改谁，互相完全不干扰，这就是所谓的“サービスごとの設定ファイル（每个服务各自的配置文件）”。

14. 虽然 `firewall-cmd` 是 CentOS 7 后的现代工具，但题干明确指出要“利用 netfilter（Linux 内核网络过滤器）”来直接进行规则设定。在 Linux 的进化史上，直接基于、并作为 `netfilter` 官方原配应用层管理工具的代名词命令，永远是老牌的 `iptables`。

netfilterを利用してパケットフィルタリングのルール設定を行うコマンドは `iptables`

要完美 get 到这道题的因果关系，我们必须理清 Linux 内部防火墙的**三层沙漏模型**：

- 幕后大 boss —— `netfilter`（内核层）**：它是住在 Linux 内核（Kernel）深处的代码，是真正动手去拦截、放行数据包的“执行硬件”。它在应用层没有命令，你没法直接敲它。
- 正牌大兵 —— `iptables`（应用层命令，本题核心）**：在 Linux 的传统设计里，`iptables` 是跟 `netfilter` 一起配套诞生的官方命令行工具。你敲的每一条 `iptables -A INPUT ...`，都是**点对点、直接且毫无保留地**翻译成底层 `netfilter` 能听懂的硬核规则。
- 现代职业经理人 —— `firewalld` 与 `firewall-cmd`（高层抽象）**：到了现代，大家觉得直接写 `iptables` 规则太烧脑了，而且重启会断网。于是引入了 `firewalld`。
 - **它的本质是**：你把指令给 `firewall-cmd` → 传递给 `firewalld` → `firewalld` 在后台把你的话转译成 `iptables` 或者 `nftables` 的语法 → 最终才下发给 `netfilter`。
 - 也就是说，现代工具和 `netfilter` 之间隔着一层大管家。
- **考场判定逻辑**：当题目强调“利用 netfilter（内核机制）直接去操纵、设定パケットフィルタリング（数据包过滤规则）的底层经典命令”时，出题人的标准指向永远是那个跟它深度绑定的老搭档 —— `iptables`！

15. `inetd.conf` 的正统配置格式一共有 7 个标准字段，其雷打不动的顺序为：**服务名** → **套接字类型** → **协议** → **是否等待(wait/nomwait)** → **运行用户** → **守护进程绝对路径** → **命令名与参数**。

在旧时代的 `inetd` 体系里，因为所有人都要挤在 `/etc/inetd.conf` 这唯一一张大表格里，所以系统对字段的位置和顺序要求极其严苛，绝对不能多一个或者少一个。

这一行合法的配置必须要满足以下 **7 大核心金刚** 的完美排队：

- サービス名（服务名）**：如 `ftp` 或 `telnet`，必须在系统 `/etc/services` 里有记录。
- ソケットタイプ（套接字类型）**：通常 TCP 走 `stream`，UDP 走 `dgram`。
- プロトコル（协议）**：一般就是 `tcp` 或者 `udp`。
- wait/nomwait（是否等待）**：【考场最高频大坑！】，决定是单线程（`wait`）还是多线程并发（`nomwait`）。
- 実行ユーザ（运行用户）**：通常是由权力最大的 `root` 来启动。
- デーモンプログラムのパス（绝对路径）**：比如 `/usr/libexec/ftpd`，必须以 `/` 开头。

g. 引数（命令名与参数）：重复一下进程名，后面可以带上参数（如 `ftpd -l`）。

老旧门卫 `inetd`，手下小弟排成排——

- 【名字（1）流（2）协议（3）】 → 前面三位是基础，服务、stream 和 tcp；
- 【中间卡着个排队鬼（4）】 → 第 4 位必须是 `wait` 或者 `nowait` ——
 - 如果没有 `wait/nowait`（比如选项1），当场打叉枪毙它；
- 【老板（5）带路径（6）】 → `root` 老板跟在后面，手里拿着绝对路径（以 `/` 开头的程序路径）；
- 【屁股后面报名字（7）】 → 最后再重复一遍进程名当结尾！

“名字流协议，`wait`（第四项）站中间，老板拿路径，秒选第二项！”

 考试小秘籍

- a. “肉眼找 `nowait` 锚点”律：这种选择题是典型的干扰视觉题。
- 秘籍：拿到题目后，第一眼直接去数每个选项里的第 4 个单词是什么！
 - 如果第 4 个单词不是 `wait` 或 `nowait`，这个选项 100% 是错的（比如第一项的第四个词是 `root`，第三项第四个词是 `root`，通通秒杀淘汰）。利用这个“第 4 位黄金锚点法”，你连后面那一长串路径都不用看，瞬间就能勾出第二项这个唯一合法的正解！

 队列位置	 正牌字段职能	 常见的合法填空长啥样？	 考场背后的运维大白话
1	服务名称	ftp / telnet	“你要替哪个小弟服务看门？”
2	套接字类型	stream（对应TCP） / dgram	“你是细水长流还是数据包？”
3	网络协议	tcp / udp	“走哪条交通网络？”
 4	wait / nowait	nowait（高频） / wait	“人多了要不要在门口排队？”
5	执行用户	root	“用谁的权限在后台干活？”
 6	绝对路径	/usr/libexec/ftpd	“真正干活的程序在哪个具体的盘符里？”
7	命令与参数	ftpd -l / telnetd	“最后把程序名字和参数再报一遍。”

16. 这道题考查的是 Linux 系统管理中，修改了传统超级服务器 `inetd` 的配置文件（`/etc/inetd.conf`）后，如何让系统重新加载（再読み込み）配置并使其生效的核心命

`inetd` 在修改配置后，既可以通过传统的服务大总管命令进行重启（`restart`），也可以通过发送 `SIGHUP`（挂断/重载信号，信号编号为 1）通知 `inetd` 进程重新读取配置文件，常见的手段有重启脚本、`kill -HUP` 和 `killall -HUP`。

当管理员修改了 `/etc/inetd.conf` 之后，新规则只是保存在硬盘里，内存里的 `inetd` 进程根本不知道。我们需要通过以下两种流派去“点醒”它：

 流派一：正统重启法（简单粗暴）

- `/etc/init.d/inetd restart`：我们在前面的旧时代服务管理中中学过，`/etc/init.d/` 是前朝 SysVinit 存放服务脚本的大本营。调用这个脚本并传入 `restart` 参数，系统会把旧的 `inetd` 彻底杀掉，然后重新拉起一个全新的进程，新配置自然就生效了。

2 流派二：信号重载法（高级不流血，考场大核心！）

在 Linux 中，很多守护进程（Daemon）都支持一种极其高级的操作：**不重启进程，只重新读取配置文件**。这个秘密武器就是发送 **SIGHUP 信号（挂断信号，名字叫 HUP）**。当 `inetd` 收到这个信号时，它会身体一震，立刻重新把 `/etc/inetd.conf` 读一遍，而现有的网络连接完全不会断开！

- `kill -HUP inetd` 的 PID：针对特定 PID 发送 HUP 信号。使用时需要配合 `ps` 查出进程号（如 `kill -HUP 1234`）。
- `killall -HUP inetd`：偷懒神器。不需要查 PID，直接根据**进程的名字（`inetd`）**，把 HUP 信号无差别精准投喂给所有叫 `inetd` 的进程。

17. `inetd` 的配置文件 `/etc/inetd.conf` 是以行为单位进行解析的，如果不想让某个服务启动（起動させたく無い），最正统、最简单的方法就是在该服务所在行的最开头加上“井号（`#`）”进行注释，这样系统读取时就会自动跳过这一行。

在 Linux 的世界里，除了极少数像 `xinetd` 那样使用 `disable = yes` 开关的特例之外，对于绝大多数**按行读取的传统配置文件**（比如 `/etc/inetd.conf`、`/etc/fstab`、`/etc/resolv.conf`），想要让某一行规则失效，根本不需要费劲去删掉它。

- **井号（`#`）的威力**：只要你在某一行代码的最前端敲下一个 `#`，这一行在系统眼里就会瞬间变成普通的“空气（注释文本）”。
- 当 `inetd` 重新加载配置时，它的扫描器只要看到行首是 `#`，就会**连看都不看一眼，直接跳过去扫描下一行**。
- 结果就是：被加了 `#` 的那个网络服务（比如 `telnet`）就**绝对不会被拉起和启动**，完美达到了“起動させたく無い”的目的。

18. 超级服务器（如 `inetd`，`xinetd`）的本职工作是替各种利用率低的小服务代管网络端口，以节省系统资源。而负责记录“IP地址与主机名对应关系（IPアドレスとホスト名の対応）”的，那是大名鼎鼎的本地解析文件 `/etc/hosts`，跟超级服务器的配置文件毫无瓜葛！

为了让你在考场上能够一眼看穿所有正确和错误的选项，我们用一个极为形象的职场画面来解释什么是超级服务器：

- **为什么要发明它？（解决资源浪费）**：如果服务器上运行着 50 个很少有人访问的小服务（比如 `tftp`、`telnet`），让这 50 个进程天天挂在内存里死等客户，会极大地**浪费系统的 CPU 和内存资源（システムリソースの無駄遣い）**。

- 它是怎么工作的？（代管请求）：于是系统派出了一个全职的“大楼保安”——超级服务器（デーモン名字叫 `inetd` 或 `xinetd`）。那 50 个小弟服务全部处于关机睡觉状态，由超级服务器一个人在门口同时监听这 50 个服务的网络端口（利用頻度の低いサービス等への要求を受け付ける）。
- 瞬间唤醒：一旦有客户敲门（比如发起 telnet 连接），超级服务器就会立马把睡觉的 `telnetd` 叫醒（拉起进程），把客户交给他。等客户走了，`telnetd` 又会关闭进程继续睡觉。通过这种方式，系统内存被优化得极其完美。

19. `xinetd` 拥有极为强悍的内置原生安全防火墙功能，它使用 `only_from` 来规定“只允许哪些 IP/网段访问”（白名单），使用 `no_access` 来规定“拒绝哪些 IP/网段访问”（黑名单）。

与前朝老臣 `inetd` 必须依赖外部的 TCP Wrappers（即 `/etc/hosts.allow` 和 `deny`）不同，新一代大总管 `xinetd` 自己就在配置文件里自带了两个顶级的门卫参数，专门负责安全审查：

a. 绝对白名单 —— `only_from`（本题核心）：

- 字面意思：Only from（只来自于...）。
- 运维大白话：系统会严格执行“非请勿进”的铁律。在这个参数后面写了谁的 IP，就只有（only）这些人能连进来，其他人统统在大门口被枪毙。
- 写法示例：`only_from = 192.168.1.0/24`（只允许这个内网段访问）。

b. 绝对黑名单 —— `no_access`（本题核心）：

- 字面意思：No access（不准访问 / 禁止入内）。
- 运维大白话：专门用来拉黑坏人。在这个参数后面写了谁的 IP，这些人就绝不（no）能获得访问权。
- 写法示例：`no_access = 203.0.113.5`（彻底封杀这个恶意攻击的 IP）。

 考试小秘籍

i. “from 与 access 绝不交叉”铁律：在做 `xinetd` 这道多选题时，有一个极其震撼的规律：

- 秘籍：正解的两个单词里，一个是以 `from` 结尾，另一个是以 `access` 结尾，它们绝对不会使用相同的词尾！
- 秒杀法：当你看到选项里有 `only_from` 时，说明以 `from` 结尾的白名单已经占了坑，那么黑名单那一个就绝对不可能叫 `deny_from` 或者 `allow_from`，它必须要换成以 `access` 结尾的词！而代表否定的词在 `xinetd` 里是最简单的 `no`。利用这个词尾分工律，你能在 2 秒钟之内在选项里把正确的一对黄金搭档直接活捉！

20. 如果想查看“当前默认区域”的全部规则，使用的是 `--list-all`；但如果想把系统里“所有区域”的规则按区域各自拉个大清单一览无遗，使用的正统选项是 `--list-all-zones`。

在 `firewall-cmd` 的查看规则体系中，主要分为两大查看流派：

1 查单家（指定区域或默认区域）：`--list-all`

- **运维大白话场景：**“我只想看看当前正在掌权的默认区域（比如 `public`），或者我指定的某一个区域（如 `--zone=dmz`）里面放行了什么服务。”
- **输出内容：**它只吐出一个特定区域的详细规则。

2 查全家（所有区域的信息）：`--list-all-zones`（本题核心）

- **运维大白话场景：**“我想做一次彻头彻尾的安全大检查，把系统自带的 `public`、`dmz`、`trusted`、`drop` 等所有 9 个区域的具体规则信息，全部挨个打印出来让我对齐。”
- **输出内容：**它会把全系统所有的区域（All Zones）逐个列个清单（List），输出信息量非常庞大。

`--list-all` ❌【死因：它默认只查一个区域，范围小了】：

- **大白话真相：**很多同学因为前几题刚背过 `--list-all`，一兴奋就秒选了这一项。记住！如果不加任何前缀，`--list-all` 只会显示默认区域（Default Zone）的信息，根本不会显示其他区域的信息，不符合题干中“全てのゾーン”的要求。

`--permanent` ❌【死因：它是“永久生效”的修改开关，不是查看选项】：

- **大白话真相：**这个选项是用来把临时规则写入硬盘的（比如 `firewall-cmd --add-service=http --permanent`），让规则开机不丢失。它本身不具备任何查看和打印规则的功能。

暗号化によるデータの保護

1. 这道题考查的是加密与数字签名工具 `gpg`（GNU Privacy Guard）中，用来对文件进行数字签名（署名），从而让别人能够验证文件是否被篡改（改ざんされていないか確認）的核心配置选项。在 `gpg` 中专门用来“生成数字签名（对文件进行加密盖章）”的选项是 `--sign`；而用来“验证签名、检查文件是否被篡改”的选项才是 `--verify`

在网络安全中，为了防止文件在传输过程中被黑客偷偷修改（改ざん），我们要使用 GPG 钥匙对进行“双向奔赴”的数字签名操作：

1 步骤一：作者在本地“盖章” → `--sign`（本题核心）

- **运维大白话场景：**“我是软件发布者，我写好了一个安装包。为了证明这是我写的、且中途没有被下毒，我要用我的私钥（Private Key）对这个文件进行数字签名（署名する）。”
- **敲下命令：**`$ gpg --sign filename`
- **结果：**生成一个带有你专属防伪印章的签名文件。

2 步骤二：用户在远端“验章” → `--verify`

- **运维大白话场景：**“我是下载软件的用户。我拿到了安装包和签名，我要确认文件在下载过程中有没有残缺或被篡改（確認できるように）。”

◦ 敲下命令： `$ gpg --verify filename.gpg`

◦ 结果：系统会明确告诉你“签名完好（完好无损）”或者是“签名失效（中途被篡改）”。

4. 这道题考查的是安全外壳协议命令 `ssh` 中，在进行公开密钥认证（公開鍵方式のユーザ認証）时，用来明确指定使用哪一个“私钥文件（秘密鍵のファイル）”的核心配置选项。

使用 `ssh` 命令手动指定私钥文件时，既可以使用经典的简写命令行参数 `-i`（Identity 的缩写），也可以在选项参数 `-o` 后面直接调用系统的标准配置项 `IdentityFile`。

我们在使用 SSH 远程登录服务器时，如果不指定钥匙，系统默认会去用户的家目录下寻找找 `~/.ssh/id_rsa` 或 `~/.ssh/id_ed25519` 等默认私钥。但如果你有多个私钥（比如公司服务器用 A 钥匙，自己家里的电脑用 B 钥匙），你就必须在命令行里主动告诉 `ssh` 命令你要掏出哪把钥匙：

在 SSH 的官方标准字典里，私钥文件（秘密鍵）不叫 Private Key，它被官方赋予了一个非常高大的代名词——叫做“身份证明文件（Identity File）”。因此：

a. 经典快捷方式 ➡ `-i`（本题核心）：

- 英文本意：Identity（身份）。
- 写法示例： `$ ssh -i ~/.ssh/my_company_key user@host`
- 大白话：这是最常用的简写，`-i` 后面直接跟着你的私钥路径。

b. 正规配置文件写法 ➡ `-o IdentityFile`（本题核心）：

- 英文本意：`-o` 代表 Option（临时调用一个标准的 SSH 配置项），而官方在配置文本里的正规参数名就叫 `IdentityFile`。
- 写法示例： `$ ssh -o IdentityFile=~/.ssh/my_company_key user@host`
- 大白话：这跟直接敲 `-i` 的底层效果完全一模一样，只不过一个是快捷键，一个是完整语法名。

◦ `ssh [オプション][ユーザ名@]接続先ホスト`

オプション	説明
-l	ログインユーザ名の指定 (本オプションと[ユーザ名@]の どちらも省略された場合は現在の ユーザ名を使用)
-p	接続先ポート番号の指定
-i	秘密鍵のファイルを指定
-o	ssh_configで設定できるオプションの指定

`ssh` 命令的基本书写格式就像寄一封安全挂号信：首先通过 `-l` 指定收件人，通过 `-p` 找到对方家门，通过 `-i` 带上自己的身份证（私钥），最后通过 `-o` 临时附带一些高级定制参数。

`-l`（Login User） ➡ 对应日文：ログインユーザ名の指定（指定登录用户名）

- p (Port)** → 对应日文：接続先ポート番号の指定（指定连接的端口号）
- i (Identity File)** → 对应日文：秘密鍵のファイルを指定（指定私钥/秘密键文件）
- o (Option)** → 对应日文：ssh_configの設定オプションの指定（临时指定高级配置项）

登录老王敲小 **-l**，找准端口快快 **-p**。 ** 秘密钥匙认准 **-i**，高级配置画个 **-o**

5. 这道题考查的是 SSH 密钥对生成（Key Generation）中常用的几种加密算法（Encryption Algorithm）及其分类（对称加密 vs 非对称加密）的辨析。

SSH 使用的是非对称加密（公开密钥方式）。在这一体系中，RSA 是经典老将，而 ECDSA 和 ED25519 是现代高性能算法，它们均属于公开密钥方式，且在相同安全性前提下，比 RSA 的运算速度更快、密钥长度更短。

SSH 只要是“指定密钥生成”的，通通都是“公开密钥方式”！

RSA 是祖师爷，ECDSA 和 ED25519 是后起之秀（快且短）。

凡是看到 DES，直接给它贴上“古董、对称加密”的标签，绝不搞混！

算法名	方式分类	考场定位 (Role)
RSA	公开密钥	经典老牌，兼容性好，但密钥较长
ECDSA	公开密钥	现代高速，密钥短，效率高
ED25519	公开密钥	目前最推荐 ，最快最安全
DES/AES	共通键	用于会话加密，而非身份认证

考试小秘籍

在 SSH 考题中，只要看到 **ED25519** 或 **ECDSA**，脑子里立刻关联“公开密钥方式”和“比 RSA 快”。只要看到 **DES**，立刻把它从 SSH 认证的选项中剔除。这套逻辑在所有技术认证考试中都是通用的！

6. `gpg` コマンドは GnuPG という暗号化プログラムを使用して、ファイルの暗号化や復号を行います。

書式: gpg [オプション]	
オプション	説明
--gen-key	公開鍵暗号方式の鍵ペアを生成する
--export	公開鍵をエクスポートする
--import	公開鍵をインポートする
--list-keys	公開鍵を一覧表示する
--sign	ファイルに署名する
--verify	ファイルの署名を検証する
-e ファイル名	暗号化データを受け取る側の公開鍵を使用してファイルを暗号化する
-o ファイル名	出力ファイル名を指定する
-r メールアドレス	暗号化に使用する公開鍵の持ち主のメールアドレスを指定する

💡 GPG 间谍特工记忆法

想象你是一名特工，要给总部发送加密情报：

第一阶段：准备工作（创建与管理钥匙）

- `--gen-key`：“根基”——要发报，先打好根基（生成密钥对）。
- `--list-keys`：“列出钥匙”——看看包里有哪几把钥匙。
- `--import` / `--export`：“进口” / “出口”——导入别人的钥匙，导出自己的钥匙。

第二阶段：加密发送（情报传递）

- `-e`：“e（一）起加密”——使用公钥把文件加密（Encrypt）。
- `-r`：“r（人）”——指定这把公钥的拥有者（Recipient，收件人）。
- `-o`：“o（输出）”——指定 Output（输出文件名）。
- 口诀：“E（加密）-R（收件人）-O（输出名）”是加密三部曲。

第三阶段：防伪与核实（签名与校验）

- `--sign`：“签名”——像给情报盖个戳。
- `--verify`：“微（ve）力（ri）验证”——只有用微小的力气验证一下，看看印章有没有被改动。

“根基（gen-key）生钥匙，导入（import）导出（export）看（list）；

加密（-e）找收件人（-r），输出（-o）文件名；

签名（sign）盖个戳，验证（verify）防改动。”

7. `ssh` 命令在远程连接时，指定“以哪个用户身份进行登录”的三种标准写法。

在 `ssh` 登录时，你可以通过 `-l` 参数指定，也可以通过 `用户名@主机名` 的格式指定，还可以通过在 SSH 配置文件（`-o`）中指定 `User` 关键字来完成。

在 Linux 运维的战场上，如果你不写用户名，SSH 默认会拿你本地电脑当前的名字去远程尝试，这往往会失败。所以，学会这三种“自报家门”的方法是基础中的基础。

a. 参数法（`-l`）：这是最直观的命令行参数。`-l` 代表 Login User。

■ 写法：`ssh -l root 192.168.1.1`

b. 格式法（`user@host`）：这是业界最通用的写法，也是大家平时敲得最多的。

■ 写法：`ssh root@192.168.1.1`

c. 高级配置法（`-o User=...`）：这是一个硬核用法。当某些脚本需要强行覆盖默认登录用户时，会用到 `-o`（Option）来临时指定 `User` 这个配置项。

■ 写法：`ssh -o User=root 192.168.1.1`

【第一招：带个名牌（`-l`）】 ➡ 门卫说：请出示你的工牌（L-ogin）；

【第二招：写在门牌上（`user@host`）】 ➡ 格式必须是：人（user）在（@）地（host），千万别把地址写到人前面去了！

【第三招：给门卫写纸条（`-o User=`）】 ➡ 用 `-o` 给门卫临时递个纸条，上面写着 `User=...`；

“记住 `-l` 和 `user@`，外加 `-o` 给纸条，三招连用稳稳拿分！”

8. 这道题考查的是 SSH 公开密钥认证（公開鍵方式）中，密钥文件的命名规则以及密钥对生成工具的使用方法

SSH 认证的核心逻辑是“公钥放服务器，私钥留本地”；文件的命名中，以 `.pub` 结尾的是公开的公钥；而生成这套密钥对的工具正是 `ssh-keygen`。

在 SSH 的世界里，密钥对如同“锁和钥匙”。我们通过 `ssh-keygen` 这台“造锁机”生成一对文件，它们的命运截然不同：

a. “造锁机”工具：`ssh-keygen`。无论你是要给用户生成登录密钥，还是要给服务器生成主机识别密钥，统统用它。

b. 文件名规则：

- 公钥（Public Key）：必须以 `.pub` 结尾。例如 `id_rsa.pub`，它是可以随处分发、甚至贴在服务器门上的。
- 私钥（Private Key）：没有 `.pub` 后缀。例如 `id_rsa`，它是你的“传家宝”，绝对不能泄露，更不能传给服务器。

	SSHクライアント	SSHサーバ(sshd)
設定ファイル	/etc/ssh/ssh_config	/etc/ssh/sshd_config
秘密鍵	~/.ssh/id_rsa	/etc/ssh/ssh_host_rsa_key
公開鍵	~/.ssh/id_rsa.pub ②	①/etc/ssh/ssh_host_rsa_key.pub
認証鍵リスト	~/.ssh/known_hosts	~/.ssh/authorized_keys
①ホスト認証時に登録 ②ユーザ認証時に登録		

💡 SSH 远程求婚故事（记忆主线）

第一阶段：签合同（配置文件）

- **客户端**：去别人家求婚，要遵守基本的社交礼仪，所以用 `ssh_config`（配置文件名比较短，因为是散客）。
- **服务器**：守护着整个家，要把关严，所以用 `sshd_config`（多了一个 `d`，代表 Daemon/守护进程，配置更复杂）。

第二阶段：随身证件（密钥）

- **客户端（你是求婚者）**：
 - 秘密（秘密键）：`id_rsa` —— 这是你自己的身份，藏在兜里（`~/.ssh/`）。
 - 公开（公开键）：`id_rsa.pub` —— 这是你的简历，随身带着准备递给对方（对应图中的②，用户认证时登记）。
- **服务器（它是被求婚者）**：
 - 秘密（秘密键）：`ssh_host_rsa_key` —— 这是服务器的家门钥匙，由系统保管（`/etc/ssh/`）。
 - 公开（公开键）：`ssh_host_rsa_key.pub` —— 这是服务器的名片，贴在门口（对应图中的①，主机认证时登记）。

第三阶段：黑名单与白名单（认证键列表）

- **客户端（知道谁是好人）**：`known_hosts`。我“已知（Known）”的主机名单，防止误入黑店。
- **服务器（知道谁能进门）**：`authorized_keys`。我是“授权（Authorized）”的用户清单。只要你的公钥在里面，你就能开门。

“客户短名（`ssh_config`）守礼仪，服务端守护（`sshd_config`）锁门严；

自持证件 `id_rsa`，房东守家 `host_key`；

公钥简历 `.pub` 递，已知（known）主机靠客户端；

授权（authorized）钥匙服务器，一进一出两清单。”

クライアントがSSH接続を行う際に必要になる、サーバの公開鍵が登録されるファイルは
`~/.ssh/known_hosts`

SSHサーバ（sshd）の設定ファイルは`/etc/ssh/sshd_config`

SSHサーバの公開鍵/秘密鍵が保存されるディレクトリは`/etc/ssh/`

💡 记忆推理：文件名中的“密码”

在 Linux 中，SSH 的配置文件名遵循一个非常严谨的逻辑。我们通过拆解文件名来排除掉所有错误选项：

a. 目录锁定 (`/etc/ssh/`) : 所有 SSH 的全局配置 (无论是客户端还是服务器) 都住在 `/etc/ssh/` 这个文件夹里。所以, 开头不是 `/etc/ssh/` 的选项直接淘汰 (比如 `/etc/sshd_config` 错误, 因为 `/etc/` 下面不会直接放这些具体的配置文件)。

b. `sshd` vs `ssh` 的区别:

- `sshd`: 字母 `d` 代表 **Daemon (守护进程)**。凡是 `sshd` 开头的, 全是**服务器端**的配置。
- `ssh`: 没有 `d`, 代表这是**客户端**的配置。

c. 组合逻辑:

- 服务器配置: `/etc/ssh/` + `sshd_config` = `/etc/ssh/sshd_config`
- 客户端配置: `/etc/ssh/` + `ssh_config` = `/etc/ssh/ssh_config`

9. SSH 公开密钥认证 (公钥方式) 的核心安全原则, 即: “**私钥 (秘密键) 必须永远留在自己的本地机器, 绝对不能发送给服务器; 而公钥 (公开键) 才是可以发给服务器或在网络间传输的。**”

那么如何把私钥文件安全地传输回本地电脑

SSH 公钥认证的黄金法则:

- a. 私钥 (Private Key, 不带 `.pub`) : 是你的数字身份证, **绝对禁止**离开生成它的本地机器。
- b. 公钥 (Public Key, 带 `.pub`) : 是你的名片, 可以发送给任何你需要连接的服务器。
- c. 生成位置: 应该在**客户端**生成。如果误在服务器上生成了密钥对, 你想把私钥拿到客户端使用, 这在安全操作上是**极度错误且危险的**。

私钥的定义: 在 SSH 中, `id_ecdsa` (不带 `.pub` 的) 是**私钥 (Private Key/秘密键)**。

验证逻辑:

- 公钥 (`.pub`) 是放在服务器的 `authorized_keys` 里的, 它本身就存在于服务器上, 不需要“持ち運ぶ” (携带/搬运) 给客户端。
- 客户端要登录服务器, 必须持有与服务器上公钥对应的**私钥**。
- 因此, 如果你必须从服务器搬运东西回客户端, 那唯一需要搬运的、且必须保密的, 就是**私钥文件**。

10. 这道题考查的是 **GnuPG (GPG)** 工具链中, 专门负责**密钥管理 (秘密键管理)** 以及**缓存密码 (Passphrase)** 状态的守护进程程序。GPG 体系中, 那个负责默默在后台帮你保管密钥、让你**不用每次加密都输密码 (缓存 Passphrase)** 的“管家”程序, 名字就叫 `gpg-agent`。

在 GPG 的工作流程中, 为了安全性, 我们需要在操作时输入 Passphrase (密钥密码)。但如果每次加密文件都要输密码, 工作效率极低。于是, GPG 引入了“管家”:

- `gpg-agent` 的职责:
 - i. 缓存 Passphrase: 你输过一次密码后, `gpg-agent` 会在内存里记住它, 并在一段时间内 (默认几小时) 不再让你输入。

ii. **秘密键管理**：它在后台守护，负责把秘密键安全地加载到内存中供调用，而不是让每一个 `gpg` 进程都去碰触硬盘上的原始密钥文件。

iii. **兼容性**：它能够模拟 `ssh-agent` 的协议，让你能用 GPG 密钥来做 SSH 登录认证。

11. 这道题考查的是 **GnuPG (GPG)** 及现代加密技术中最为核心的两种加密方式——公开密钥方式（非对称加密）与共通键方式（对称加密）的底层逻辑差异。

“共通键方式”就像是一把通用钥匙，双方要用同一把锁；而“公开密钥方式”就像是“邮筒”：任何人都可以把信投进你的邮筒（公钥加密），但只有你手里有钥匙能打开邮筒取信（私钥解密）。

1 公开键方式（非对称加密 - 如 GPG/SSH）

- 原理：你有两把钥匙，一把公开的（公钥），一把私有的（私钥）。
- 逻辑：“加密用公钥，解密用私钥”。这就像邮筒：任何人都可以用你的公钥把信件“锁死”发给你，但除了你自己，没有任何人能用私钥把它“打开”。
- 安全性：极高，解决了“密钥怎么安全传给对方”的问题。

2 共通键方式（对称加密 - 如 AES/DES）

- 原理：发送方和接收方使用完全相同的一把锁和钥匙。
- 逻辑：发送前双方必须先商量好这把密钥。
- 安全性风险：如果要把这把钥匙传给对方，万一中途被截获，所有的加密就全完了。所以，为了降低风险，你应该针对不同的联系人，分别使用不同的钥匙。如果所有的联系人都用同一把钥匙，一旦其中一个联系人把钥匙弄丢了，所有人都会面临风险。

12. 公开键方式（公开密钥/非对称加密）：

- 特点：使用一对密钥（公钥加密，私钥解密）。
- 常见成员：**RSA、ECDSA、ED25519**。

共通键方式（对称加密）：

- 特点：加密和解密使用同一把钥匙。
- 常见成员：**DES、AES**。

“公开密钥（RSA, ECDSA, ED25519）是大明星，成双成对；DES 是老古董，自己管自己（共通键）。”

极简谐音口诀：“公开（公钥）是双星（成对），DES（得饿死）是单丁（对称）。”

在密码学中，这些算法各有其独特的“出生背景”和“看家本领”：

- RSA**：它是公开密钥加密的“祖师爷”。在 1977 年左右由 **Ron Rivest、Adi Shamir 和 Leonard Adleman** 三人提出，是历史上第一个被广泛实用的公开密钥加密算法。它的安全性建立在“大整数的素因数分解”这一数学难题之上。

- **ECDSA**：全称是 *Elliptic Curve Digital Signature Algorithm*（**椭圆曲线**数字签名算法）。选项中提到的“エドワーズ曲線（Edwards Curve）”是 ED25519 的特性，不是 ECDSA 的定义。
- **ED25519**：这是目前**最先进、安全性最高且速度最快**的签名算法之一，绝不是“安全性最低”。
- **SSH**：这是一个**网络协议（Protocol）**，不是一个加密算法，更不是那个基于素因数分解的算法（那是 RSA 的原理）。

💡 算法身份锚点口诀：

- **[RSA]** ➡ “**第一名**”：它是最早出现的公开密钥加密算法（素因数分解是它的招牌）。
- **[ECDSA]** ➡ “**椭圆曲线**”：看到 ECDSA 脑子里必须立刻出现“椭圆曲线”四个字。
- **[ED25519]** ➡ “**现代王者**”：看到它，脑子里要打上“最快、最强”的标签。

“**RSA 是祖师爷（实用的先驱），ECDSA 是椭圆曲线，SSH 是大协议，别把原理当协议！**”

9. 这道题考查的是 **SSH 公开密钥认证（公開鍵方式）** 的完整工作流程，这是 LPIC 考试中关于 SSH 安全最核心的考点。

SSH 公钥认证的本质是：客户端将自己的公钥存入服务器的 `authorized_keys`，连接时服务器用这个公钥核对客户端私钥生成的签名，证明你确实拥有该私钥。

为了保证安全，SSH 必须分两步走：首先确定“主机没错”，然后确定“用户没错”。

a. 主机认证（防止中间人攻击）：

- 客户端连接服务器时，为了确认对方不是冒牌货，客户端会将服务器的公钥（Host Key）存入自己的 `~/.ssh/known_hosts`。这样下次连接时，客户端就会核对对方是不是那个“老熟人”。

b. 用户认证（身份证明）：

- 用户将自己的公钥（User Key）上传并存入服务器的 `~/.ssh/authorized_keys`。服务器通过这个文件来“授权（Authorized）”哪些人可以进门。

区分“谁存谁”：在做这类题目时，先画一个十字表：

- `known_hosts` ➡ 客户端存服务器（为了辨别对方）。
- `authorized_keys` ➡ 服务器存客户端（为了放行用户）。

“**私钥永不离身**”：记住，任何选项里提到“服务器持有用户的秘密键”或“对比私钥”，直接排除，这违背了公钥加密的数学原理。

这道题的陷阱在于它精准地考查了“**主机认证（Host Authentication）**”和“**用户认证（User Authentication）**”这两个完全不同的概念。

选选项 5（ユーザの公開鍵をサーバの「`~/.ssh/authorized_keys`」に登録しておく）是错的，原因在于：这句话描述的是“**用户认证**”，而不是“**主机认证**”。

💡 核心区分：主机认证 vs 用户认证

在 SSH 连接的整个过程中，有两个维度的“身份核实”：

a. 主机认证 (Host Authentication) —— “我是不是那个正确的服务器？”

- 目的是什么？防止你连到了黑客伪装的假服务器（中间人攻击）。
- 动作：客户端查看服务器的公钥，并将其存入 `~/.ssh/known_hosts`。
- 结论：这是一个主机认证动作。

b. 用户认证 (User Authentication) —— “服务器认不认识我这个用户？”

- 目的是什么？确认你是否有权进入这台服务器。
- 动作：用户将自己的公钥上传，存入服务器的 `~/.ssh/authorized_keys`。
- 结论：这是一个用户认证动作。

题目问的是“主机认证（ホスト認証）”的说明。

◦ 选项 5 (`ユーザの公開鍵をサーバの「~/.ssh/authorized_keys」に登録しておく`)：

这是在配置用户权限，属于用户认证的范畴。

虽然这确实是 SSH 建立连接后的重要步骤，但它不属于所谓的“主机认证 (Host Authentication)”。主机认证是在用户登录之前发生的，用于校验“机器身份”。

 考场排查表：别被关键字骗了！

动作类型	核心关键字	涉及文件	参与方
主机认证	主机名、指纹、钓鱼、假冒	known_hosts	客户端核对服务器
用户认证	权限、登录、身份、授权	authorized_keys	服务器核对客户端

 小学生级记忆法（过关安检口诀）

 3秒 认证区别口诀：

- 【过海关第一关：看门牌（主机认证）】  我得确认这是不是我要找的那个“家”，所以我查我的通讯录 `known_hosts`。
- 【过海关第二关：查护照（用户认证）】  家里人得确认我是不是受邀的客人，所以他们查他们的授权清单 `authorized_keys`。

“主机认证看门牌（known_hosts），用户认证查名单（authorized_keys）。别搞反了！”

 考试小秘籍

在 LPIC 或相关考试中，出题人非常喜欢把这两个概念混在一起考。

- 如果题目出现“ホスト認証（主机认证）”，选项里只要看到 `authorized_keys`，直接排除。
- 如果题目出现“ユーザ認証（用户认证）”，选项里只要看到 `known_hosts`，直接排除。

10. SSH 端口转发的命令行格式非常统一，只要掌握了这个模板，就不会出错：

```
ssh -L [本地端口]:[目标主机]:[目标端口] [用户名]@[SSH服务器]
```

- **-L (Local Port Forwarding)**: 意思是“把本地的流量转发出去”。这正是题目要求的“本地端口 8000 连接到远程 Web 服务”。

- **参数含义:**

- 8000 (本地端口)**: 是你自己在电脑上用来“敲门”的端口。
- localhost (目标主机)**: 站在 SSH 服务器的视角, Web 服务就在它自己身上, 所以是 `localhost`。
- 80 (目标端口)**: Web 服务运行的实际端口。

选项中的 -R: 这是“远程端口转发 (Remote Port Forwarding)”。题目要求的是“本地连接”, 所以必须选 **-L**。凡是带 **-R** 的选项全部直接排除!

端口位置颠倒: 端口转发的顺序是“先本地后目标”。如果写成 `80:localhost:8000`, 意思就变成了“把本机的 80 端口转发到远程的 8000 端口”, 这完全错了。

地址格式错误: 像 `ssh-server:80` 这种写法是错误的, SSH 命令的最后一部分必须是 `用户名@主机名`。

11. SSH 的密码 (Passphrase) 是用来加密你的私钥文件的。为了安全性, 这个密码一旦设置, 系统绝对不会以任何明文形式存储它, 更不会提供任何“显示”或“找回”密码的功能。

忘れてしまったパスワードを表示・消去する方法は無い (没有办法显示或清除已经忘记的密码)

在 SSH 的世界里, 你的私钥文件 (例如 `id_rsa`) 本质上是一个被“上锁”的加密容器。

- **加密逻辑**: 当你设置 Passphrase 时, SSH 会使用这个密码通过加密算法 (如 AES) 对私钥内容进行加密, 并存储在硬盘上。
- **找回逻辑的缺失**: Linux 系统 **不会** 保存你的密码, 也不会保存密码的“提示”。如果你忘了密码, 意味着你没有“钥匙”来解开那个加密容器, 因此系统根本无法还原原始的私钥内容。

12. Linuxで通常使われるSSHのオープンソースは**OpenSSH**

13. GnuPGに関連するファイルの置かれるディレクトリ名 `~/.gnupg/`

14. 为了安全和隐私, GnuPG 所有的个人设置 (包括你的私钥、公钥环、信任数据库) 都存放在用户家目录下的隐藏文件夹 `.gnupg` 中。

15. 在 GPG 中, 如果你收到了一个加密文件 (通常以 `.gpg` 结尾), 要把它还原成原本的内容, 只需要直接调用 `gpg` 命令即可。GPG 会自动检测文件类型, 识别到加密内容后, 会提示你输入密码进行解密, 并将结果输出。👉 `gpg hogehoge.txt.gpg`

16. 这道题考查的是 SSH 公开密钥认证中最核心的**安全性原则**: **绝对不能外泄私钥 (秘密键)**。

在 SSH 的世界里, 身份认证的逻辑是:

- 私钥 (Private Key, 如 `id_rsa`)**: 是你的数字身份证, **必须严格保密**, 绝对不能通过邮件、聊天软件或直接交给他人。一旦私钥被他人获取, 你的账户就等于完全暴露。

b. 公钥 (Public Key, 如 `id_rsa.pub`): 是可以公开分享的名片。服务器通过持有你的公钥, 来验证你是否拥有对应的私钥。

在本题的场景中, 用户将**私钥**发送给了管理员, 这意味着私钥已经泄露了, 该密钥对已不再安全。在 Linux 考试中, 凡是遇到“把私钥传给别人”或者“私钥在网络上传输”的选项, 直接把它当成“零分答案”。

记住: 公钥 (`.pub`) 是可以满世界飞的“名片”, 私钥 (没有 `.pub` 的) 是永远不能见光的“底牌”。

17. sshコマンドの-XオプションについてDISPLAY環境変数が自動で設定される

当你使用 `ssh -X` 连接远程服务器时, 它的核心目的是让你可以在远程服务器上运行图形界面程序 (GUI 应用), 并将这个程序的窗口通过 SSH 隧道投射到你本地的显示器上。

为了实现这个投射, SSH 协议需要做一件非常重要的事情: 它必须在远程服务器的 shell 中自动设置好 `DISPLAY` 环境变量, 让远程的 GUI 程序知道“往哪里画图”。

- 【`-X` 是干嘛的?】 ➡ `X = X Window` (图形界面)。
- 【它是怎么做到的?】 ➡ 帮你把远程的“画板” (GUI 程序) 搬回本地, 所以它会自动帮你设置好指路牌 (`DISPLAY` 环境变量)。

“`-X` 带 GUI, 窗口飞回来; `DISPLAY` 环境变, 隧道全自动!”

关键词锁定: 在考试中, 只要看到 `-X`, 脑子里必须第一时间出现 `DISPLAY` 这个词。它们是“绑定关系”。

“环境”的精准性: 记住它是“环境变量 (Environment Variable)”, 考试里故意用“shell 变量”来干扰你, 这是典型的陷阱, 请务必选“环境变量”。

18. 这道题考查的是 SSH 客户端命令中最常用的选项之一: 如何修改默认的连接端口 (SSH 默认端口是 22, 但有时服务器会修改为其他端口, 如 2222)。

在 SSH 的世界里, 虽然命令非常多, 但修改端口的语法是比较“特别”的:

- a. `-p`: 这是最直接、最通用的短格式选项。
- b. `-o Port=xxx`: 这是 SSH 的“万能配置接口” (`-o` 代表 Option), 通过 `Port=xxx` 的方式来指定。这在某些自动化脚本或高级配置中非常有用。

19. 認証エージェントであるssh-agentに秘密鍵を登録するコマンドはssh-add

`ssh-agent` 的作用是作为一个后台服务, 帮你“记住”已经解密的私钥。这样你只需要在第一次使用密钥时输入一次密码 (Passphrase), 之后该会话内就不需要重复输入。而将私钥“交给”这个管家保管的命令, 就是 `ssh-add`。

ssh-agentに登録された秘密鍵の一覧を表示するコマンドは`ssh-add -l`

命令	功能	考点关键字
ssh-agent	启动代理守护进程	起動、バックグラウンド、デーモン
ssh-add	添加私钥到代理	登録、秘密鍵、パスフレーズ保持
ssh-add -l	列出已添加的密钥	確認、リスト表示
ssh-add -D	清除所有已添加的密钥	削除、全消去

20. 当你需要通过一台“跳板机”（中间服务器）连接到另一台内网服务器时，通常的做法是将密钥拷贝到跳板机上。但这样极其不安全！**SSH Agent Forwarding** 允许你利用本地电脑的 `ssh-agent` 来认证，不需要把私钥传给任何中间服务器。实现这一功能的唯一关键参数就是 `-A`。

sshコマンドで「-A」オプションを使用して接続する

💡 3秒 Agent Forwarding 转发口诀：

- 【它是谁？】 ➡ `-A` = Agent 转发（跳板机的“传声筒”）。
 - 【怎么记？】 ➡ **A** for Agent Forwarding。
 - 【铁律】 ➡ 别去 `ssh -a`（小写的 `-a` 往往是关闭代理），一定要用 `-A`（大写）开启转发！
- “大 `A` 转发能接力，跳板机器显神技；无需密钥放中转，代理通道最安全！”